Preparazione all'interview
Assignment 2 – 2AMM40

Machine Learning for Science – Probabilistic Simulation

Teoria del corso, collegamenti teoria–pratica, codice spiegato,
scelte fatte e alternative, risposte ANS commentate, banca domande

Formato dell'interview (dal messaggio del docente): max 15 minuti, tutti i membri del gruppo presenti in persona. Domande su (i) contenuto degli assignment, (ii) contenuto del corso e soprattutto (iii) *le risposte che avete dato in ANS*. Possono chiedere di *motivare una scelta di modellazione* oppure *come cambierebbe la soluzione se cambiassero i requisiti*. Lo scopo: verificare che abbiate raggiunto i learning outcome e che *capiate la vostra soluzione*.

Slot: lun 19 ottobre 12:45–18:45 (Atlas 3.32) oppure mer 21 ottobre 8:45–14:45 (Atlas 8.310). Consegna ANS: venerdì 16 ottobre, 23:59.

Legenda dei box

Teoria	definizioni e formule del corso
Teoria ↔ pratica	dove la teoria entra nella vostra soluzione
Da dire (EN)	frasi pronte, in inglese, per l'interview
Punto debole	cose su cui il docente può insistere
Da sistemare	incoerenze da correggere in ANS prima del 16/10

Fonti: slide lezioni 00–04, testo Assignment 2, notebook `Assignment_2_part_1_1.ipynb` (codice e output), risposte ANS del gruppo. La parte 2 si basa solo sulle risposte ANS (il notebook della parte 2 non era disponibile).

Contents

1 Come usare questo documento e cosa sistemare subito **3**
1.1 Piano di studio consigliato (lun 19 / mer 21) . . . 3
1.2 Incoerenze trovate confrontando ANS e notebook . . . 3

2 La teoria del corso, organizzata per l'interview **5**
2.1 Lezioni 00–01: sistemi dinamici e neural surrogates . . . 5
2.2 Lezione 02: Geometric Deep Learning e sistemi Euleriani . . . 6
2.3 Lezione 03: sistemi Lagrangiani e limiti dei modelli autoregressivi . . . 7
2.4 Lezione 04: surrogate probabilistici, transport models, flow matching . . . 8
2.5 Tre derivazioni da saper fare alla lavagna . . . 9

3 Parte 1 – Il problema: CylinderFlow su mesh non uniforme **11**
3.1 Dati (numeri esatti dal notebook) . . . 11
3.2 La fisica che dovete saper raccontare . . . 11
3.3 La metrica: wake temporal activity . . . 12

4 Parte 1 – La vostra soluzione, codice spiegato **14**
4.1 Visione d'insieme . . . 14
4.2 Cella 18 – Iperparametri . . . 14
4.3 Cella 19 – Da mesh a grafo, padding e batching . . . 14
4.4 Cella 20 – Dati in RAM e statistiche di normalizzazione . . . 15
4.5 Cella 21 – Feature di input . . . 15
4.6 Cella 22 – La rete . . . 16
4.7 Cella 23 – Loss e sampler . . . 17
4.8 Cella 24 – Training loop . . . 18
4.9 Celle 36–37 – Adapter e rollout . . . 19

5 Parte 1 – Risultati e come interpretarli **20**
5.1 Curve di loss . . . 20
5.2 Wake temporal activity (test, 20 traiettorie) . . . 21
5.3 Rollout visivi . . . 22
5.4 Lo sweep sui passi di Euler (cella 41): cosa è successo . . . 23

6 Parte 1 – Scelte fatte, alternative non fatte, "what if" **25**
6.1 Tabella delle scelte . . . 25
6.2 "How would your solution change if . . . " . . . 26

7 Parte 1 – Le vostre risposte ANS, commentate **28**
7.1 2a–2c: CNN, Transformer, GNN . . . 28
7.2 3a–3b: perché un modello probabilistico . . . 28
7.3 4a: architettura del deterministico . . . 29

7.4 4b: embedding del tempo di flusso τ . . . 30
7.5 4c: condizionamento e differenze tra i due modelli . . . 30
7.6 5a–5b: procedura di training . . . 30
7.7 5c–5d: curve di loss . . . 31
7.8 5e: numero di passi di integrazione . . . 31
7.9 6b, 6d, 6e: GIF qualitative . . . 31
7.10 6g: analisi della metrica . . . 32
7.11 7a–7b: bozze pronte (le risposte sono vuote in ANS) . . . 32

8 Parte 2 – Pedoni a Eindhoven Centraal **33**
8.1 Il problema in breve . . . 33
8.2 La vostra soluzione (dalle risposte ANS) . . . 33
8.3 Teoria del prior informato (11b): perché dovrebbe aiutare e perché V1 non ha aiutato 34
8.4 Risultati (14a) . . . 34
8.5 Metriche distribuzionali (14b) . . . 35
8.6 11a: bozza (la risposta è vuota in ANS) . . . 35
8.7 Altri punti caldi della parte 2 . . . 36

9 Confronto tra le due parti (domande “trasversali”) **37**

10 Banca domande (risposte brevi in inglese) **38**
10.1 Teoria generale del corso . . . 38
10.2 Parte 1: architettura e dati . . . 39
10.3 Parte 1: flow matching e training . . . 40
10.4 Parte 1: risultati . . . 40
10.5 Domande trabocchetto . . . 41

11 Cheat sheet (da rileggere prima di entrare) **42**

1 Come usare questo documento e cosa sistemare subito

1.1 Piano di studio consigliato (lun 19 / mer 21)

1. **Oggi–giovedì:** sistemare le incoerenze di ANS nella sezione 1.2 (la consegna è venerdì 16 alle 23:59, e il docente usa ANS come "ground truth" durante l'interview).
2. **Teoria (sez. 2):** saper *derivare* a voce tre risultati: (a) l'MSE porta alla media condizionata; (b) la loss di conditional flow matching ha lo stesso ottimo della loss "ideale"; (c) un solo passo di Euler dal rumore restituisce la media condizionata. Sono le tre idee su cui si regge tutto l'assignment.
3. **Parte 1 (sez. 3–7):** ogni membro deve saper disegnare l'architettura alla lavagna, spiegare ogni feature di input, la loss, il sampler e i risultati.
4. **Parte 2 (sez. 8):** stesso discorso per il modello dei pedoni (prior gaussiano vs prior informato).
5. **Ultimo giorno:** banca domande (sez. 10) a voce, in inglese, a turno. Poi rileggete il cheat sheet (sez. 11).

Punto debole / trappola: regola d'oro

Se non sapete una cosa, ditelo e ragionate ad alta voce partendo dalla teoria ("we did not test this, but we would expect … because …"). Il docente valuta la *comprensione*, non la perfezione dei risultati. Mai inventare un esperimento che non avete fatto: per esempio lo sweep sul numero di passi di Euler *non* è stato eseguito (vedi sotto).

1.2 Incoerenze trovate confrontando ANS e notebook

Ho confrontato riga per riga le risposte ANS con il codice e gli output del notebook. Questi punti vanno corretti in ANS (o almeno vanno preparati per l'interview).

Da sistemare prima della consegna: risposte vuote

- **7a** (limitazioni non ovvie, parte 1) e **7b** (miglioramenti di efficienza, parte 1) risultano *vuote* ("0 words"). Propongo bozze pronte nella sez. 7.11.
- **11a** (perché un modello probabilistico per i pedoni e in cosa i dati differiscono dalla parte 1) è *vuota*. Bozza in sez. 8.6.
- Le domande **8**, **15**, **16** non erano nel testo che mi hai girato (probabilmente upload del notebook, dichiarazione d'uso di AI, contributi). Controllate che siano compilate.

Da sistemare prima della consegna: errori fattuali o di forma

1. **5a, hardware.** ANS dice "Michael's laptop with a CUDA GPU". Il notebook stampa `Device: mps`, cioè GPU Apple (Metal), non CUDA. Il crash dello sweep dice "MPS backend out of memory". Correggete in "Apple-silicon laptop GPU (PyTorch MPS backend)". In 5b la frase "same laptop … with CUDA" va cambiata allo stesso modo.
2. **5a, tempo.** La domanda chiede "the time it took" e la risposta non lo dice. Dal log: 7–8 s per epoca, 30 epoche, quindi circa 4 minuti, uguale al modello FM.
3. **5e, "collapses to the data mean".** È impreciso: un solo passo di Euler da $\tau = 0$ restituisce la *media condizionata* $\mathbb{E}[\Delta\mathbf{u} \mid \mathcal{C}_t]$, cioè quello che stima il modello deterministico, non la media dei dati. La derivazione è nel box 2.5. Lo dice anche il commento del vostro codice (cella 41).
4. **5e, esperimento.** Lo sweep sui passi di Euler (cella 41) è andato in *out of memory* su MPS e non ha prodotto output. In più la funzione `rollout_activity_validation` è chiamata ma non è definita da nessuna parte, quindi `activity_vs_steps()` fallirebbe comunque.

La risposta 5e è quindi solo teorica. Va bene, ma all'interview va detto chiaramente. Se avete tempo prima di venerdì, rilanciate solo `one_step_vs_steps()` con meno batch (vedi sez. 5.4).

5. **10b, segnaposto dimenticato.** Il testo contiene letteralmente "connecting everyone *[used / would use]* too much GPU memory". Scegliete una delle due.
6. **13b, "pd:".** Probabilmente volevate scrivere "PS:".
7. **3a, "in a chaotic system such as this".** A questi numeri di Reynolds la scia di von Kármán è soprattutto *periodica* (ciclo limite), non caotica in senso stretto. La risposta regge lo stesso (sensibilità della *fase*, informazione persa dalla discretizzazione, regime che cambia tra traiettorie), ma preparatevi la versione sfumata nel box 3.2.
8. **9a/13a, "first 10,000 samples".** L'assignment chiede uno split *casuale a livello di sample*. Se avete preso i primi 10k e poi li avete divisi con seed 42, chiarite che lo split è casuale *all'interno* di quei 10k e preparate la motivazione (budget di calcolo su T4). La 14e dice "since we have plenty of data": coerente, ma il docente può chiedere perché ne avete usato solo il 10%.

Punto debole / trappola: due numeri che possono generare domande scomode (parte 1)

1. **Validation loss del deterministico $\approx$ 2.5.** Con target normalizzati (media 0, varianza 1 per componente) e una loss che somma le due componenti, un predittore costante ("incremento medio") ottiene circa 2 sulla distribuzione di training. Il vostro miglior deterministico ha 2.52 in validation. Questo significa che, sulla validation, la predizione puntuale a 1 s non batte (o batte appena) il predittore banale. Non è necessariamente un errore: è proprio il messaggio dell'assignment (dopo 1 s la media condizionata contiene poca informazione). Però va saputo spiegare. Vedi sez. 5.1 e lo snippet per calcolare la baseline.
2. **Gap train/validation già all'epoca 1** (1.68 vs 5.21 deterministico; 1.80 vs 3.20 FM). L'overfitting classico si sviluppa *nel tempo*. Un gap presente da subito indica anche una differenza di distribuzione tra le 20 traiettorie di validation e il training (per esempio regimi di flusso con incrementi più grandi), oltre alla stima rumorosa su sole 20 traiettorie.

2 La teoria del corso, organizzata per l'interview

Questa sezione ripercorre le lezioni 00–04 seguendo il filo che serve per l'Assignment 2. Per ogni blocco c'è il collegamento esplicito a cosa avete fatto voi.

2.1 Lezioni 00–01: sistemi dinamici e neural surrogates

Teoria: sistema dinamico e soluzione numerica

Stato $u \in U$ (variabili che descrivono completamente il sistema), parametri θ fissi, legge di evoluzione $\frac{du}{dt} = \Phi(u, t)$. La soluzione è

$$u_t = u_0 + \int_0^t \Phi(u_s, s)\, ds.$$

Senza soluzione analitica si integra numericamente. Con Euler esplicito $u_{t+1} = u_t + \Delta t\, \Phi(u_t, t)$, quindi l'ODE diventa una *sequenza autoregressiva*. Per una PDE lo stato è un *campo*: $u(t) \to u(x, t)$, $\partial_t u = \mathcal{F}(u)$ (es. equazione del calore $\partial_t u = \alpha \nabla^2 u$, Navier–Stokes per il cylinder flow).

Teoria: neural surrogate

Una rete f_ϕ addestrata su dati di simulazione che approssima la mappa di evoluzione:

- **frame-by-frame (autoregressivo):** $u_{t+1} = f_\phi(u_t)$, poi $u_{t+2} = f_\phi(u_{t+1}), \ldots$
- **time bundle:** $u_0 \mapsto u_{1:T}$ in una sola chiamata;
- **sequence-to-sequence:** $u_{0:T} \mapsto u_{T+1:2T}$;
- **neural operator:** $u_0 \mapsto u(t)$ come funzione continua del tempo.

Perché conviene? Il costo dell'addestramento viene *ammortizzato* su molte valutazioni, e il surrogate può fare passi molto più lunghi del solver ("computational reducibility", lezione 02: campo visivo più ampio, rappresentazione più efficiente). Le tre domande guida del corso per ogni problema sono: *qual è il dominio dei dati? quali sono le simmetrie? quale modello è adatto?*

Teoria: Euleriano vs Lagrangiano

Euleriano: osservo i campi in punti *fissi* dello spazio, lo stato è $u(x, t)$ (cylinder flow: velocità su una mesh fissa). Discretizzazione regolare (griglia) o non regolare (mesh).
Lagrangiano: seguo elementi materiali che si muovono, lo stato è $u(X(t), t)$ e le coordinate fanno parte dello stato dinamico (particelle, N-body, pedoni). Dati irregolari, non ordinati, geometricamente strutturati: insiemi o grafi con connettività che cambia nel tempo.

Teoria ↔ pratica

Parte 1 = sistema **Euleriano su mesh non uniforme**: i nodi sono fissi, cambia solo la velocità. Parte 2 = sistema **Lagrangiano**: le posizioni dei pedoni *sono* lo stato, il numero di agenti varia, il grafo va costruito (kNN). In entrambe le parti il modello è un *surrogate condizionale*. Nella parte 1 è autoregressivo con passo $\Delta t_{\text{pred}} = 1$ s (100 volte il passo dei dati). Nella parte 2 è un time bundle da 20 frame.

2.2 Lezione 02: Geometric Deep Learning e sistemi Euleriani

Teoria: simmetria, invarianza, equivarianza

Una simmetria è una trasformazione che lascia invariata una proprietà del sistema. Per un gruppo con azione g:

$$\text{invarianza: } f(\rho_x(g)\,x) = f(x), \qquad \text{equivarianza: } f(\rho_x(g)\,x) = \rho_y(g)\,f(x).$$

Se *ogni* layer h_i commuta con il gruppo, $h_i(\rho_{i-1}(g)x) = \rho_i(g)h_i(x)$, allora la composizione $f = h_L \circ \cdots \circ h_1$ è equivariante. Per costruire layer equivarianti: gli operatori lineari devono commutare con il gruppo. Le attivazioni standard vanno bene su feature scalari; per feature vettoriali/tensoriali servono attivazioni specializzate.
Perché conta: **data efficiency**, **consistenza** sotto cambi di coordinate, **meno predizioni non fisiche**. Esempi: griglie → traslazioni; molecole → permutazioni e rotazioni/traslazioni; cristalli → simmetria periodica.

Teoria: convoluzione = equivarianza per traslazione

$(K * x)_{i,b} = \sum_{r\in S}\sum_a K_{r,a,b}\,x_{i-r,a}$ e $(K * (T_\delta x)) = T_\delta(K * x)$. Una somma (pooling globale) sopra un layer equivariante dà un'*invarianza*. Per la CFD su griglia: la U-Net è equivariante per traslazione. Il campo di velocità è un campo *vettoriale*, però una traslazione non cambia la base dei vettori ($\mathbf{u}(x-a) = (u_x(x-a), u_y(x-a))$), quindi la U-Net resta corretta. Una *rotazione* invece ruoterebbe anche le componenti: la convoluzione standard non è equivariante per rotazione.

Teoria: message passing (Gilmer et al.) e equivarianza per permutazione

$$x_i^{(k)} = \gamma^{(k)}\Big(x_i^{(k-1)}, \bigoplus_{j\in\mathcal{N}(i)} \phi^{(k)}\big(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}\big)\Big)$$

ϕ = funzione di messaggio (la stessa per ogni arco), γ = update del nodo (la stessa per ogni nodo), $\bigoplus$ = aggregazione invariante per permutazione (somma, media, max). Quindi l'MPNN è **equivariante per permutazione** dei nodi. Un readout globale ($z = \sum_i x_i$) lo rende invariante. k iterazioni $\Rightarrow$ campo recettivo di k salti.

Teoria: l'attenzione è equivariante per permutazione (senza positional encoding)

$Q = HW^q$, $K = HW^k$, $V = HW^v$, $\text{Att} = \text{softmax}(QK^\top/\sqrt{d_k})V$. Con $\tilde{H} = PH$: $\tilde{Q}\tilde{K}^\top = PQK^\top P^\top$ e $\text{softmax}(PSP^\top) = P\,\text{softmax}(S)P^\top$ (softmax per riga). Quindi $\text{Att}(PH) = P\,\text{Att}(H)$.
Attenzione: equivarianza per permutazione ≠ equivarianza geometrica. Un GNN o un transformer *non* commutano automaticamente con le trasformazioni euclidee (rotazioni, traslazioni) delle coordinate.

Teoria: MeshGraphNets (Pfaff et al., 2021)

Encoder–processor–decoder su grafo della mesh. Nel processor, per ogni arco si aggiorna prima la feature d'arco da (arco, mittente, destinatario), poi il nodo da (nodo, somma dei messaggi). MGN originale usa 15 step di message passing con connessioni residue. Per alcuni domini (es. tessuti) aggiunge anche *world edges*; per CylinderFlow bastano i *mesh edges*. Il decoder dà una derivata p_i e lo stato si aggiorna come un passo di Euler:

$$q_i^{t+1} = q_i^t + \Delta t\, p_i \qquad (\Delta t = 1 \text{ nella slide}).$$

È stato introdotto proprio per CylinderFlow, con passo dei dati 0.01 s. Risultati: MGN batte una U-Net su griglia uniforme, perché la mesh concentra la risoluzione dove serve.

Teoria ↔ pratica: cosa avete preso da qui

- Mesh → grafo, archi = lati dei triangoli in entrambe le direzioni: struttura di MGN.
- Feature d'arco = vettore relativo $x_j - x_i$ e sua norma: rende *il message passing* invariante per traslazione (principio ripreso anche nella lezione 03).
- Predizione dell'incremento $\Delta\mathbf{u}$ e somma allo stato attuale: è l'analogo dell'"Euler step" di MGN, $q^{t+1} = q^t + p$.
- Attenzione globale: permutation-equivariant come il message passing, quindi l'intero modello è equivariante per permutazione dei nodi. Rinumerare i nodi della mesh permuta l'output allo stesso modo. Le posizioni assolute entrano come *feature del nodo*, non come encoding basato sull'indice, quindi non rompono questa proprietà.

2.3 Lezione 03: sistemi Lagrangiani e limiti dei modelli autoregressivi

Teoria: GNN geometrici e principio della posizione relativa

Un MLP che riceve coordinate assolute $\phi(\ldots, x_i, x_j)$ non ha nessun motivo architetturale per dare lo stesso risultato dopo una traslazione $x \mapsto x + g$. Usare lo spostamento relativo $r_{ij} = x_i - x_j$ elimina la dipendenza dalla traslazione, perché $(x_i + g) - (x_j + g) = x_i - x_j$. Però dare le *componenti* di r_{ij} a un MLP non garantisce l'equivarianza per rotazione.

Teoria: E(n)-equivariant GNN (Satorras et al., 2021)

$$
\begin{aligned}
m_{ij} &= \phi\big(h_i, h_j, \|x_i - x_j\|^2, e_{ij}\big) && \text{(messaggi invarianti)} \\
x_i^{(k+1)} &= x_i^{(k)} + C \sum_{j \neq i} (x_i - x_j)\, \zeta(m_{ij}) && \text{(scalare} \times \text{direzione geometrica)} \\
h_i^{(k+1)} &= \gamma\Big(h_i^{(k)}, \sum_{j \neq i} m_{ij}\Big) && \text{(embedding invarianti)}
\end{aligned}
$$

Due principi di design: (1) costruire feature *invarianti* dalle quantità fisiche (distanze); (2) aggiornare le quantità vettoriali come *coefficiente scalare per direzione geometrica*. Prova: con $\tilde{x} = Qx + g$ i messaggi non cambiano e l'update dà $Qx^{(k+1)} + g$.

Teoria: errore composto nei modelli autoregressivi

Con $\hat{x}_t = f_\theta(\hat{x}_{<t})$ ed errore $e_t = \hat{x}_t - x_t$, l'espansione di Taylor dà

$$
\hat{x}_t \approx f_\theta(x_{<t}) + J_t\, e_{<t}, \qquad J_t = \frac{\partial f_\theta}{\partial x_{<t}}, \qquad e_t \approx J_t\, e_{<t}.
$$

L'errore cresce in proporzione alla *sensibilità del modello all'input*. Nel caso stocastico $e_t \approx J_t e_{<t} + \xi_t$. Per un modello lineare semplice $e_{t+1} = (1 + \alpha) e_t + \mathcal{N}(0, \sigma^2)$ e, se $\alpha \approx 0$, si ha un random walk con $\mathrm{RMSE}(e_t) \approx \sigma\sqrt{t}$.

Teoria: teacher forcing, push-forward, time bundling

Teacher forcing: in training l'input è sempre lo stato *vero*: $L = \|f_\theta(x_t) - x_{t+1}\|^2$. Il modello non vede mai i propri errori (*exposure bias*).

Push-forward (Brandstetter et al. 2022): l'input è la predizione del modello stesso, $L = \|f_\theta(\hat{x}_t) - x_{t+1}\|^2 = \|f_\theta(x_t + e_t) - x_{t+1}\|^2$ (di solito senza gradiente attraverso il primo passo). Il modello impara a correggere la distribuzione dei propri errori.
Time bundling: si predicono T passi in una volta. Ci sono meno chiamate, quindi meno passi su cui l'errore si compone. Il costo è un compito più difficile e un orizzonte fisso.

Teoria ↔ pratica

Nella parte 1 avete usato **teacher forcing** puro: le coppie (u_t, u_{t+1s}) vengono tutte dal simulatore. Nessun push-forward e nessun rumore sugli input (MGN originale aggiunge rumore agli input proprio per questo). Questo è uno dei motivi per cui nei rollout alcuni campioni FM "si spengono": dopo qualche passo l'input è uno stato leggermente fuori distribuzione e il modello non l'ha mai visto. Ottima "limitation" per la 7a. Nella parte 2 avete usato il **time bundling**, che evita del tutto la composizione degli errori dentro i 20 frame.

2.4 Lezione 04: surrogate probabilistici, transport models, flow matching

Teoria: perché probabilistico

Sistemi caotici o fortemente non lineari (doppio pendolo, N-body) hanno alta sensibilità alle condizioni iniziali: piccole incertezze sull'input diventano grandi incertezze sull'output, e le bande di confidenza si allargano nel tempo. Con condizioni iniziali piccole il doppio pendolo è invece quasi deterministico: non tutti i sistemi sono ugualmente incerti. Messaggio: *modellare gli esiti plausibili, non solo la loro media*. Il surrogate diventa un modello generativo condizionale $x_{t+1} \sim p_\theta(x \mid x_t)$, e campionare equivale a simulare.

Teoria: obiettivo generativo e autoregressive models

Massima verosimiglianza $\theta^\star = \arg\max_\theta \mathbb{E}_{p_{data}} \log p_\theta(\mathbf{x})$, che equivale a $\arg\min_\theta D_{KL}(p_{data} \| p_\theta)$. ARM (fattorizzazione per componenti): $p_\theta(\mathbf{x}) = \prod_i p_\theta(x_i \mid \mathbf{x}_{<i})$ (PixelCNN, Row LSTM, attenzione causale). Limiti: campionamento lento in alta dimensione, difficoltà con dipendenze a lungo raggio, accumulo di errore durante il campionamento.

Teoria: transport model e perché la regressione diretta fallisce

Si vuole $T_\theta : \mathbb{R}^d \to \mathbb{R}^d$ tale che $\mathbf{x}_0 \sim p_0 = \mathcal{N}(0, I)$ e $T_\theta(\mathbf{x}_0) \sim p_{data}$. Se lo si imposta come regressione supervisionata $\min_\theta \mathbb{E}\|T_\theta(X_0) - X_1\|^2$, l'ottimo è

$$T^*(x_0) = \mathbb{E}[X_1 \mid X_0 = x_0] = \mathbb{E}[X_1] \qquad \text{se } X_0 \perp X_1.$$

Non c'è accoppiamento tra rumore e dato, quindi ogni rumore va nella *media*: la distribuzione collassa. Serve un compito "a livello di popolazione" (population-level task).

Teoria: normalizing flows e CNF

Cambio di variabili: $\log p_X(x) = \log p_Z(f^{-1}(x)) + \log \left| \det \partial f^{-1}/\partial x \right|$. Vincoli: invertibilità, Jacobiano costoso, dimensione costante, espressività limitata.
Continuous NF: $\frac{dX_t}{dt} = v_\theta(X_t, t)$, $X_t = \phi_t(X_0)$, $p_t = (\phi_t)_\# p_0$. L'evoluzione della densità segue l'**equazione di continuità**

$$\partial_t p_t(x) + \nabla \cdot \big(p_t(x)\, v_t(x)\big) = 0.$$

Divergenza positiva espande il volume e abbassa la densità, negativa la concentra. Addestrare una CNF per massima verosimiglianza richiede di risolvere ODE (anche per $\log p$ con la traccia

dello Jacobiano), quindi è lento e instabile. ResNet = passo di Euler di una neural ODE: $x_{n+1} = x_n + h\,v(t_n, x_n)$.

Teoria: flow matching

Obiettivo ideale: $\mathcal{L}_{FM} = \mathbb{E}_{t, X_t \sim p_t} \|v_\theta(X_t, t) - u_t(X_t)\|^2$, ma u_t e p_t non sono noti.
Percorsi condizionali affini gaussiani: $X_t = \alpha_t X_1 + \sigma_t \epsilon$ con $\alpha_0 \approx 0, \sigma_0 \approx 1, \alpha_1 \approx 1, \sigma_1 \approx 0$, e $u_t(X_t \mid X_1) = \dot{\alpha}_t X_1 + \dot{\sigma}_t \epsilon$.
Percorsi rettilinei ($\alpha_t = t, \sigma_t = 1 - t$): $X_t = (1-t)X_0 + tX_1$, $U_t = X_1 - X_0$.
Conditional flow matching: scelgo un *coupling* $(X_0, X_1) \sim \pi$ (indipendente: $\pi = p_0 \otimes p_1$), $t \sim \mathcal{U}[0,1]$, e

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{(X_0, X_1) \sim \pi,\, t} \left\| v_\theta\big((1-t)X_0 + tX_1,\, t\big) - (X_1 - X_0) \right\|^2.$$

Training *simulation-free*: niente ODE in training, solo una regressione. L'ODE si integra solo in generazione: $\hat{X}_1 = X_0 + \int_0^1 v_\theta(X_t, t; c)\, dt$.

Teoria: direct regression vs flow-matching regression (slide 53, la più importante)

Diretta $X_0 \mapsto X_1$: ottimo $\mathbb{E}[X_1 \mid X_0]$, cioè la media delle *destinazioni finali*. Collassa.
Flow matching $(X_t, t) \mapsto X_1 - X_0$: ottimo

$$v^*(x, t) = \mathbb{E}[X_1 - X_0 \mid X_t = x, t].$$

X_t contiene informazione su *entrambi* gli estremi, quindi la regressione media *movimenti locali* in un punto e in un tempo dati, non destinazioni finali. Integrando l'ODE, il campione "sceglie" progressivamente un modo e non collassa.

2.5 Tre derivazioni da saper fare alla lavagna

Teoria: D1: la MSE impara la media condizionata

Per qualsiasi funzione f e target Y con condizione c, vale la scomposizione

$$\mathbb{E}\|f(c) - Y\|^2 = \mathbb{E}\|f(c) - \mathbb{E}[Y \mid c]\|^2 + \mathbb{E}\big[\operatorname{tr}\operatorname{Cov}(Y \mid c)\big].$$

Il termine misto si annulla per la tower property: $\mathbb{E}[(f(c) - \mathbb{E}[Y|c])^\top (\mathbb{E}[Y|c] - Y)] = \mathbb{E}[(f(c) - \mathbb{E}[Y|c])^\top \mathbb{E}[\mathbb{E}[Y|c] - Y \mid c]] = 0$. Il secondo termine non dipende da f. Quindi $f^*(c) = \mathbb{E}[Y \mid c]$ e la loss minima è la *varianza condizionata*, che non si può eliminare.
Nella parte 1: il modello deterministico impara $\mathbb{E}[\Delta\mathbf{u} \mid \mathcal{C}_t]$. Se dopo 1 s la fase dei vortici è incerta, la media somma vortici in posizioni diverse che si cancellano: il risultato è un campo liscio e una scia smorzata. Non è un bug, è l'ottimo della loss.

Teoria: D2: perché la CFM loss "funziona" e non va a zero

Applico la stessa scomposizione con $Y = X_1 - X_0$ e condizione (X_t, t):

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t, X_t} \left\| v_\theta(X_t, t) - v^*(X_t, t) \right\|^2 + \underbrace{\mathbb{E}\big[\operatorname{tr}\operatorname{Cov}(X_1 - X_0 \mid X_t, t)\big]}_{\text{non dipende da } \theta},$$

con $v^*(x, t) = \mathbb{E}[X_1 - X_0 \mid X_t = x]$. Si dimostra che v^* è proprio il campo di velocità *marginale* che trasporta p_0 in p_1 (soddisfa l'equazione di continuità per il percorso p_t). Quindi minimizzare la CFM loss equivale a minimizzare la loss ideale $\mathcal{L}_{FM}$ (stesso gradiente in θ).
Conseguenze per le vostre risposte 5c/5d e 13b:

- la CFM loss ha un minimo *strettamente positivo*, perché il target $X_1 - X_0$ contiene il rumore;
- il suo valore assoluto *non* si confronta con l'MSE del deterministico (oggetti diversi);
- nella parte 2 il valore dipende dal prior (V2 ha la loss più alta ma è il modello migliore);
- una loss bassa non garantisce campioni buoni: va guardata la qualità dei campioni.

Teoria: D3: un solo passo di Euler restituisce la media condizionata

Con coupling indipendente dato c, a $t = 0$ si ha $X_t = X_0$ e $X_0 \perp X_1 \mid c$, quindi

$$v^*(x, 0) = \mathbb{E}[X_1 - X_0 \mid X_0 = x, c] = \mathbb{E}[X_1 \mid c] - x.$$

Un passo di Euler di ampiezza 1 da x_0 dà

$$\hat{x}_1 = x_0 + v^*(x_0, 0) = \mathbb{E}[X_1 \mid c]$$

per *qualsiasi* rumore x_0: tutti i campioni coincidono con la media condizionata, cioè con la predizione del modello deterministico. Con pochi passi si ottiene quindi qualcosa di intermedio tra deterministico e generativo. Il motivo: i percorsi condizionali sono rette, ma il campo *marginale* v^* cambia con t, quindi le traiettorie marginali sono *curve*. Euler assume velocità costante sul passo e taglia le curve. Per lo stesso motivo le traiettorie marginali non si incrociano (l'ODE ha soluzione unica), mentre le rette condizionali sì.

Teoria ↔ pratica: come queste tre derivazioni spiegano i vostri risultati

- D1 spiega perché il deterministico ha attività di scia che crolla da 0.049 a 0.005, e perché nelle GIF diventa statico.
- D2 spiega perché la CFM loss si ferma a circa 0.46 (train) / 0.79 (val) e perché la curva FM non si confronta numericamente con quella deterministica.
- D3 spiega la risposta 5e: meno passi spingono i campioni verso la media, quindi verso il comportamento deterministico.

3 Parte 1 – Il problema: CylinderFlow su mesh non uniforme

3.1 Dati (numeri esatti dal notebook)

	train	valid	test
traiettorie	500	20	20
stati per traiettoria	600	600	600
nodi min / medio / max	552 / 601.1 / 898	561 / 593.0 / 633	565 / 586.2 / 633
coppie da 1 s	250,000	10,000	10,000

- $\Delta t_{\text{data}} = 0.01$ s, tempi $0, 0.01, \ldots, 5.99$ s. Orizzonte di predizione $\Delta t_{\text{pred}} = 1$ s, cioè 100 stati. Ogni traiettoria dà $600 - 100 = 500$ coppie. "1 secondo è la separazione tra input e target, non la spaziatura tra esempi consecutivi."
- Per traiettoria: `mesh_pos` $(N, 2)$, `cells` $(M, 3)$, `node_type` (N), `velocity` $(600, N, 2)$. La mesh è fissa nel tempo, ma cambia tra traiettorie (numero di nodi, connettività, posizione e raggio del cilindro).
- Tipi di nodo: 0 interno, 4 inflow, 5 outflow, 6 wall. **Dinamici** = interni + outflow. Inflow e wall hanno velocità prescritte e costanti, da *ripristinare* dopo ogni passo del rollout.
- Esempio (traj_0000 train): 596 nodi, 958 triangoli, 379 nodi dinamici su 596.
- Dominio: $x \in [0, 1.6]$, $y \in [0, 0.41]$ (da `pos_min`/`pos_max`). Lunghezza media degli archi nel training: 0.0371.
- **Manca la pressione:** c'è solo la velocità. Per un fluido incomprimibile la pressione è determinata dalla velocità (equazione di Poisson), quindi in linea di principio **u** è lo stato completo. Sulla mesh discreta, però, l'informazione tra i nodi è persa.

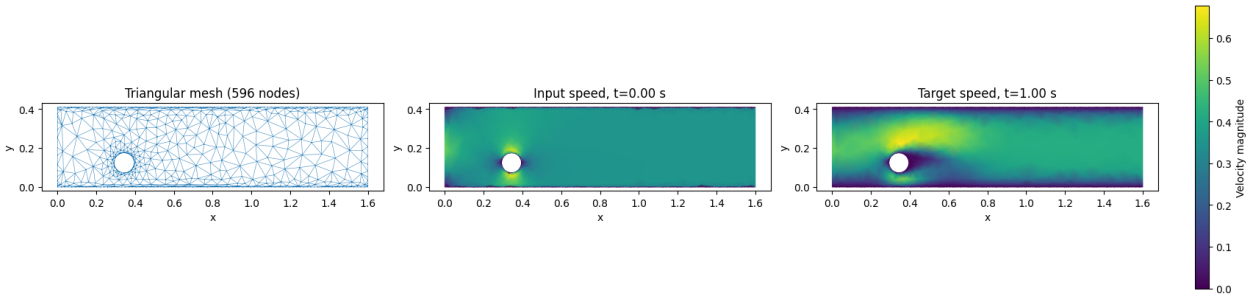


Figure 1: Mesh e una coppia di input/target a 1 s (output del notebook). La mesh è molto più fitta vicino al cilindro.

3.2 La fisica che dovete saper raccontare

- Flusso 2D in un canale attorno a un cilindro, con inflow a sinistra e outflow a destra. Sopra un certo numero di Reynolds $Re = UD/\nu$ si forma la **scia di von Kármán**: distacco periodico e alternato di vortici. Sotto, la scia è stazionaria (bolla di ricircolo). Velocità d'ingresso e geometria variano tra traiettorie, quindi il **regime cambia tra traiettorie**. Lo si vede dalla deviazione standard enorme della wake activity del ground truth (circa 0.09 su una media di circa 0.10) e dalla vostra risposta 6d (traj_0001 "more constant").
- **Ordine di grandezza** (stima, non misurata): con $U \sim 1$ m/s e $D \sim 0.1$ m, e un numero di Strouhal tipico $St = fD/U \approx 0.2$, si ha $f \approx 2$ Hz, cioè un periodo di circa 0.5 s. In 1 s passano circa due cicli di distacco. In 1 s il fluido percorre circa 1 m, cioè una frazione grande del canale lungo 1.6 m.

- **Transitorio iniziale:** lo stato a $t = 0$ non è ancora a regime. Nel vostro plot dell'attività a $t = 1$ s il ground truth cambia *ovunque*, anche a monte del cilindro. Per questo l'intervallo $0 \to 1$ s è escluso dalla metrica (burn-in).

Teoria: caos, periodicità e incertezza: la versione sfumata

Le equazioni sono deterministiche. Perché allora un modello probabilistico?

1. **Fase dei vortici.** In un regime periodico (ciclo limite) la direzione di fase è *neutralmente stabile*: un piccolo errore di fase non viene smorzato e si accumula. Dopo 1 s (circa 2 cicli) un piccolo errore sul periodo dà un'incertezza grande sulla posizione dei vortici. A Reynolds più alti la dinamica può diventare davvero caotica e l'effetto è ancora più forte.
2. **Informazione persa nella discretizzazione.** La mesh è grossolana lontano dal cilindro, quindi stati fisici diversi hanno la stessa rappresentazione sui nodi. Dal punto di vista del modello la mappa $u_t \mapsto u_{t+1s}$ non è una funzione ma una distribuzione.
3. **Errore di modello.** Ogni passo del rollout introduce un errore. Un modello che sa rappresentare un insieme di futuri plausibili resta "sulla varietà" degli stati realistici, invece di convergere al campo medio.

Quindi l'incertezza qui è soprattutto *epistemica / da osservazione parziale*, non aleatoria in senso stretto. Il modello generativo cattura la distribuzione delle fasi compatibili con quello che riesce a risolvere dello stato.

Da dire all'interview (EN): se vi chiedono "is this flow chaotic?"

"Strictly speaking, at these Reynolds numbers vortex shedding is mostly periodic, a limit cycle, rather than fully chaotic. But the phase of a limit cycle is neutrally stable, so small errors in phase are never damped. Over a 1-second jump, which is about two shedding periods, a tiny error becomes a large uncertainty in where the vortices are. On top of that, the mesh only observes the flow at a few hundred nodes, so different physical states look the same to the model. From the model's point of view the 1-second map is therefore a distribution, not a function, and that is why a probabilistic model is useful even though Navier–Stokes is deterministic."

3.3 La metrica: wake temporal activity

Per due stati a tempi t_0, t_1:

$$a_i(t_0, t_1) = \frac{\|\mathbf{u}_i(t_1) - \mathbf{u}_i(t_0)\|_2}{t_1 - t_0}, \qquad a_K = \frac{a_i + a_j + a_k}{3}, \qquad D_{\text{wake}} = \frac{\sum_{K \in \Omega_{\text{wake}}} A_K \, a_K}{\sum_{K \in \Omega_{\text{wake}}} A_K},$$

con Ω_{wake} = triangoli con baricentro in $0.55 \le x \le 1.55$, $0.03 \le y \le 0.39$.

- **Perché pesata per area:** la mesh è non uniforme. Una media sui nodi sovrappeserebbe le zone fitte (vicino al cilindro). a_K è la media esatta sul triangolo dello scalare interpolato linearmente.
- **Vettore completo:** contano sia i cambi di modulo sia quelli di direzione.
- **Burn-in:** si scartano $0 \to 1$ s; si valutano $1 \to 2, \ldots, 5 \to 6$ s. Il ground truth finisce a 5.99 s, quindi l'ultimo intervallo GT è $5.00 \to 5.99$ diviso per 0.99 s, mentre i modelli usano $5 \to 6$ diviso per 1 s.
- **Aggregazione:** per FM si calcola la metrica per *ognuno* dei 5 rollout, poi si fa la media per traiettoria e intervallo, poi media e deviazione standard sulle 20 traiettorie. **Non** si mediano i campi: mediare i campi reintrodurrebbe proprio lo smoothing del modello deterministico.

Punto debole / trappola: cosa la metrica NON misura

- Non misura l'accuratezza puntuale né la *fase* o la posizione dei vortici. Due campi con vortici in posti diversi possono avere la stessa attività.

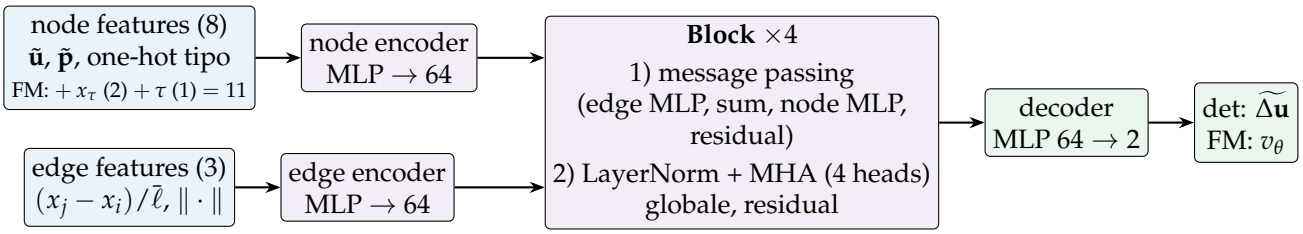
- Premia *qualsiasi* cambiamento, anche il rumore. Un modello FM che genera campi rumorosi (pochi passi di integrazione, modello sottoaddestrato) avrebbe un D_{wake} gonfiato e sembrerebbe migliore. Per questo va sempre affiancata a controlli visivi (le GIF). Nel vostro caso il FM è *sotto* il ground truth, quindi non c'è un problema di gonfiamento, ma è bene saperlo dire.
- È una statistica della dinamica ("quanto si muove la scia"), adatta a valutare un modello generativo. Un MSE contro il ground truth penalizzerebbe un campione realistico con la fase sbagliata più della media liscia, cioè premierebbe il deterministico per il motivo sbagliato (vedi D1).

Da dire all'interview (EN): "why not evaluate with MSE against the ground truth?"

"Because for a probabilistic forecast the MSE is minimised by the conditional mean. A sharp sample with vortices at a slightly wrong phase gets penalised twice, once for missing the true vortex and once for putting a vortex where there is none, while a blurred mean field gets penalised only once. So MSE would rank the deterministic model higher even though its rollouts are physically wrong. The wake activity measures a property of the dynamics instead, namely whether the wake keeps changing at the right rate, which a generative model can match without matching the exact phase."

4 Parte 1 – La vostra soluzione, codice spiegato

4.1 Visione d'insieme



Una sola classe `MeshNet` serve per entrambi i modelli. Il modello FM differisce solo per 3 canali di input in più (x_τ e τ) e per la loss. Stessa larghezza, profondità, ottimizzatore, numero di iterazioni e *stessa sequenza di coppie di training* (stesso seed numpy). Così il confronto isola "point prediction vs distribution".

4.2 Cella 18 – Iperparametri

```
HIDDEN = 64; N_BLOCKS = 4; N_HEADS = 4
BATCH_SIZE = 8; N_EPOCHS = 30; BATCHES_PER_EPOCH = 200   # 6000 steps
LR = 1e-3; WEIGHT_DECAY = 1e-4
FM_EULER_STEPS = 20
N_PAD_SMALL, E_PAD_SMALL = 640, 3584    # 488/500 train meshes fit here
N_PAD_LARGE, E_PAD_LARGE = 1024, 5632   # the few larger meshes
```

- Una "epoca" non è un passaggio sui 250k esempi, ma 200 batch casuali. In totale $6000 \times 8 = 48{,}000$ coppie viste, circa il 19% delle coppie possibili. Le coppie sono comunque fortemente correlate: quelle in t e $t + 0.01$ s sono quasi identiche.
- **Padding a due sole forme fisse:** su MPS/GPU ogni nuova forma di tensore ricompila i kernel. Con mesh di dimensione variabile ogni iterazione era lentissima; con il padding ci sono solo due forme possibili.

4.3 Cella 19 – Da mesh a grafo, padding e batching

```
def mesh_edges(cells):
    sides = np.concatenate([cells[:, [0, 1]], cells[:, [1, 2]], cells[:, [2, 0]]])
    sides = np.unique(np.sort(sides, axis=1), axis=0)   # each undirected side once
    return np.concatenate([sides, sides[:, ::-1]])       # both directions
```

- Ogni triangolo dà 3 lati; i lati condivisi da due triangoli compaiono due volte, quindi si ordinano (i, j) e si deduplicano. Poi si tengono entrambe le direzioni (sender→receiver e viceversa). I triangoli servono *solo* a costruire gli archi. Per una triangolazione planare ci sono circa $3N$ lati, quindi circa $6N$ archi diretti (un po' meno per via dei bordi: il padding 3584 per 640 nodi corrisponde a $5.6N$).
- `build_graph`: `pos`, one-hot del tipo (4 colonne nell'ordine NORMAL, INFLOW, OUTFLOW, WALL), `dyn_mask`, `src`, `dst` e `edge_vec` $= x_{\text{dst}} - x_{\text{src}}$.
- `make_batch`: impila B mesh in tensori $(B, N_{\text{pad}}, \cdot)$. I nodi di padding hanno `node_mask`$= 0$ e `dyn_mask`$= 0$. Gli archi di padding collegano il nodo 0 al nodo 0 e hanno `edge_mask`$= 0$, quindi il loro messaggio viene moltiplicato per zero.

Teoria ↔ pratica: batching: scelta vostra vs alternativa classica

L'alternativa standard (PyTorch Geometric) è il **grafo disgiunto**: si concatenano tutti i nodi del batch e si spostano gli indici degli archi di un offset. Così non c'è padding e non si sprecano calcoli. Voi avete scelto un tensore *denso e paddato* (B, N_{pad}, D) perché (i) l'attenzione di PyTorch vuole comunque un layout denso con `key_padding_mask`, e (ii) forme fisse evitano le ricompilazioni su MPS. Il costo è un po' di calcolo sprecato sui nodi fittizi (circa 600 nodi reali su 640).

4.4 Cella 20 – Dati in RAM e statistiche di normalizzazione

L'HDF5 è compresso gzip a chunk di 10 stati, quindi leggere una coppia alla volta dal disco è lento. Si caricano train e validation in RAM una volta (circa 1.5 GB) e si campionano coppie casuali:

```
def sample_train_batch(rng):
    traj = rng.integers(0, len(train_graphs), size=BATCH_SIZE)
    t = rng.integers(0, 600 - HORIZON_STEPS, size=BATCH_SIZE)   # t in [0, 499]
    return make_batch(graphs[traj], vel[traj][t], vel[traj][t + 100])
```

Validation: insieme **fisso** di coppie, cioè le 20 traiettorie a 10 tempi equispaziati $t \in \{0, 55, 110, \ldots, 499\}$, per 200 coppie e 25 batch. È sempre lo stesso insieme, quindi le loss sono confrontabili tra epoche.

statistica	valore	calcolata su	uso
`u_mean`, `u_std`	$(0.223,\ 0.0006)$, $(0.510,\ 0.115)$	tutti i nodi train	input velocità
`du_mean`, `du_std`	$(-0.0074,\ 0.0001)$, $(0.100,\ 0.064)$	solo nodi dinamici	target $\Delta\mathbf{u}$
`pos_min`, `pos_max`	$(0, 0)$, $(1.6, 0.41)$	tutti i nodi train	posizione in $[-1, 1]$
`edge_scale`	0.0371	tutti gli archi train	scala archi

- **Solo split di training**, applicate invariate a validation e test, come richiesto.
- Normalizzazione *per componente*: u_y ha una scala molto più piccola di u_x (il flusso va da sinistra a destra), quindi senza normalizzazione la loss sarebbe dominata da u_x.
- `du_std` (≈ 0.1) è circa 5 volte più piccolo di `u_std` (≈ 0.5): l'incremento a 1 s è piccolo rispetto al campo. Predire $\Delta\mathbf{u}$ normalizzato dà alla rete un target di scala unitaria. Predire $\mathbf{u}$ direttamente la costringerebbe a ricopiare la parte grande e costante del campo con precisione.
- `u_mean`/`u_std` includono i nodi di parete, dove $u = 0$. È un piccolo bias, ma è consistente tra train e test.
- Per il FM è essenziale che il target sia normalizzato: il rumore $x_0 \sim \mathcal{N}(0, I)$ e il target x_1 hanno così scale confrontabili, e i percorsi x_τ sono ben condizionati.

4.5 Cella 21 – Feature di input

```
def node_features(batch, u):                          # 8 channels
    u_n   = (u - u_mean) / u_std
    pos_n = 2 * (pos - pos_min) / (pos_max - pos_min) - 1
    x = torch.cat([u_n, pos_n, type_onehot], dim=-1)
    return x * node_mask[..., None]                   # zero on padded nodes

def edge_features(batch):                             # 3 channels
    vec = edge_vec / edge_scale
    return torch.cat([vec, vec.norm(dim=-1, keepdim=True)], dim=-1)
```

feature	perché
velocità normalizzata (2)	lo stato attuale u_t, cioè la condizione principale
posizione in $[-1, 1]$ (2)	dà all'attenzione una nozione di "dove" (l'attenzione da sola non conosce lo spazio) e permette di sapere dove sono pareti, inflow, cilindro e scia
one-hot tipo (4)	rende esplicite le condizioni al contorno
vettore d'arco / $\bar{\ell}$ (2)	geometria locale, invariante per traslazione; dà direzione e anisotropia
lunghezza d'arco / $\bar{\ell}$ (1)	risoluzione locale della mesh (fine vicino al cilindro)

Informazioni implicite: la **velocità d'inflow**, che fissa il regime e il Reynolds, si legge sui nodi di inflow; il **cilindro** (posizione e raggio variabili) si ricava dai nodi wall e dalle loro coordinate. Nessuna informazione dal futuro.

Punto debole / trappola: invarianza per traslazione: vera solo per gli archi

In 4a scrivete "Using only relative vectors on edges makes the local geometry translation-invariant". È vero per il *message passing*, ma il modello *nel suo insieme* non è invariante per traslazione, perché le posizioni assolute sono feature dei nodi. È una scelta voluta: il dominio è un canale fisso (pareti a $y = 0$ e $y = 0.41$, inflow a $x = 0$), quindi il problema *non* è invariante per traslazione. Sapere dove si è rispetto alle pareti è informazione utile, e l'attenzione ha bisogno di posizione. Non è nemmeno equivariante per rotazione: le componenti del vettore d'arco entrano in un MLP qualsiasi (lezione 03). Va bene, perché il flusso ha una direzione privilegiata ($+x$).

Da dire all'interview (EN): "is your model translation invariant?"

"Only partly, and on purpose. Message passing uses relative edge vectors, so the local operations do not depend on absolute position. But we also give the absolute, normalised node position as a node feature, so the full model is not translation invariant. We think that is correct here: the channel walls, the inflow and the wake region are at fixed locations, so the problem itself is not translation invariant, and attention needs positional information. The model is however exactly permutation equivariant: renumbering the mesh nodes just permutes the output."

4.6 Cella 22 – La rete

```
def mlp(n_in, n_out):   # as in MeshGraphNets
    return nn.Sequential(nn.Linear(n_in, HIDDEN), nn.SiLU(),
                         nn.Linear(HIDDEN, n_out), nn.LayerNorm(n_out))

class Block(nn.Module):
    def forward(self, h, e, batch):
        # 1) local message passing over mesh edges
        msg = self.edge_mlp(torch.cat([e, h[src], h[dst]], -1))
        msg = msg * edge_mask[..., None]          # padded edges send nothing
        e = e + msg                               # residual edge update
        agg = zeros_like(h).scatter_add_(1, dst, msg)    # SUM at receiver
        h = h + self.node_mlp(torch.cat([h, agg], -1))   # residual node update
        # 2) global self-attention within each mesh
        a = self.attn_norm(h)
        a, _ = self.attn(a, a, a, key_padding_mask=(node_mask == 0))
        h = h + a
        return h, e
```

In formule, per il blocco k (con h_i latente del nodo e e_{ij} latente dell'arco $i \to j$):

$$m_{ij} = \mathrm{MLP}_e\big([\,e_{ij},\, h_i,\, h_j\,]\big), \qquad e_{ij} \leftarrow e_{ij} + m_{ij},$$
$$\bar{m}_j = \textstyle\sum_{i:\,(i\to j)} m_{ij}, \qquad h_j \leftarrow h_j + \mathrm{MLP}_n\big([\,h_j,\, \bar{m}_j\,]\big),$$
$$h \leftarrow h + \mathrm{MHA}\big(\mathrm{LN}(h)\big) \quad \text{(sui nodi della stessa mesh).}$$

- **Somma** come aggregazione (come MGN). In una mesh triangolare i gradi dei nodi sono abbastanza uniformi (circa 6), quindi la somma non crea grandi squilibri; e a differenza della media conserva l'informazione su quanti vicini ci sono. Nella parte 2 avete invece usato la *media*.
- **Residual** ovunque e **LayerNorm** in uscita agli MLP: stabilità numerica con 4 blocchi e lr 10^{-3}.
- **Attenzione pre-LN**: `key_padding_mask` fa sì che nessun nodo "guardi" nodi fittizi. Ogni mesh nel batch ha la sua attenzione; mesh diverse non si parlano. Le righe di query dei nodi fittizi producono qualcosa, ma non entrano mai nella loss né nell'output.
- **Niente feed-forward dopo l'attenzione** (un blocco transformer standard avrebbe MHA + FFN). Qui l'MLP del nodo del blocco successivo ne fa le veci. Il decoder non ha LayerNorm in uscita: deve poter produrre valori di qualsiasi scala.

Teoria ↔ pratica: perché attenzione globale: il conto del campo recettivo

Con message passing puro, K step vedono K salti. Con lunghezza media degli archi $\bar{\ell} \approx 0.037$, 4 step coprono circa 0.15 (e molto meno vicino al cilindro, dove gli archi sono più corti). In 1 s il fluido percorre circa 1 unità su un canale lungo 1.6. Servirebbero decine di step: MGN usa 15 step per $\Delta t = 0.01$ s, cioè per un orizzonte 100 volte più corto. Con circa 600 nodi l'attenzione completa costa $600^2 \approx 3.6 \times 10^5$ coppie per testa e per layer, cioè pochissimo. Dà campo recettivo globale in un solo layer. L'idea è vicina a GraphGPS: message passing locale + attenzione globale in ogni blocco.

Conteggio dei parametri (verificato: 197,506 e 197,698)

modulo	parametri
node encoder MLP$(8 \to 64)$: $8\cdot 64{+}64 + 64\cdot 64{+}64 + 128$	4,864
edge encoder MLP$(3 \to 64)$	4,544
per blocco: edge MLP $(192 \to 64)$	16,640
node MLP $(128 \to 64)$	12,544
LayerNorm attenzione	128
MHA (in-proj $3\cdot 64\cdot 64{+}192$, out-proj $64\cdot 64{+}64$)	16,640
totale per blocco $\times 4$	183,808
decoder $64 \to 64 \to 2$	4,290
deterministico	**197,506**
FM: $+3$ canali in ingresso $\times 64$	$+192 \Rightarrow$ **197,698**

Il 93% dei parametri è nei 4 blocchi. In ogni blocco circa due terzi sono message passing (29,184) e un terzo attenzione (16,768). Il modello è piccolo (circa 0.2 M parametri).

4.7 Cella 23 – Loss e sampler

```
def masked_mse(pred, target, mask):        # mean over dynamic nodes of ||.||^2
    return (((pred - target) ** 2).sum(-1) * mask).sum() / mask.sum()

def deterministic_loss(model, batch):
```

```
    target = normalise_du(u_next - u_now)
    pred = model(node_features(batch, u_now), batch)
    return masked_mse(pred, target, dyn_mask)

def flow_matching_loss(model, batch, generator=None):
    dyn = dyn_mask[..., None]
    x1  = normalise_du(u_next - u_now) * dyn
    x0  = torch.randn(x1.shape, generator=generator) * dyn
    tau = torch.rand((B, 1, 1), generator=generator)       # one tau per sample
    x_tau = (1 - tau) * x0 + tau * x1
    inputs = torch.cat([node_features(batch, u_now), x_tau, tau.expand(-1, N, 1)], -1)

    v = model(inputs, batch)
    return masked_mse(v, x1 - x0, dyn_mask)
```

- La loss è *esattamente* quella dell'assignment: $\frac{1}{N_{dyn}} \sum_{i \in D} \| \cdot \|^2$, con somma sulle 2 componenti e media sui nodi dinamici.
- FM: x_1 = incremento normalizzato; rumore e target sono messi a zero sui nodi di bordo (dove $\Delta \mathbf{u} = 0$ comunque), e la loss usa solo i nodi dinamici. Un solo τ per campione (per mesh), condiviso da tutti i nodi: l'intero campo è a uno stesso "livello di rumore".
- Il `generator` serve solo a rendere la validation loss FM riproducibile (stesso rumore e stesso τ a ogni epoca).

```
@torch.no_grad()
def flow_matching_sample(model, batch, n_steps=20, generator=None):
    dyn  = dyn_mask[..., None]
    cond = node_features(batch, u_now)            # computed ONCE, fixed during the
    ODE
    x = torch.randn(u_now.shape, generator=generator) * dyn   # x at tau = 0
    dtau = 1.0 / n_steps
    for k in range(n_steps):
        tau_channel = torch.full_like(x[..., :1], k * dtau)  # left end of the step
        v = model(torch.cat([cond, x, tau_channel], -1), batch)
        x = (x + dtau * v) * dyn                  # explicit Euler
    return x                                      # sample of normalised du
```

Euler esplicito da $\tau = 0$ a 1 con 20 passi, cioè 20 valutazioni della rete per ogni passo fisico di 1 s. La condizione $\mathcal{C}_t$ è calcolata una volta; cambiano solo x_τ e τ. È lo stesso schema di `ns_sample_next` del Practical 3.

4.8 Cella 24 – Training loop

```
optimizer = AdamW(params, lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=30 * 200)  # per-step cosine to 0
for epoch in 1..30:
    for _ in range(200):
        loss = loss_fn(model, sample_train_batch(rng))
        loss.backward(); clip_grad_norm_(params, 1.0); optimizer.step(); scheduler.
    step()
    val = validation_loss(model, loss_fn)          # fixed pairs, seeded for FM
    if val < best: save checkpoint                 # model selection on validation
```

- AdamW + cosine schedule + gradient clipping (come nel Practical 3). Stesso `seed` numpy per entrambi i modelli, quindi *stessa sequenza di coppie*.
- **Model selection** sulla validation loss a un passo (best epoch: 19 deterministico, 28 FM). Nessuna decisione presa sul test set.
- Tempi reali: 7–8 s per epoca, circa 4 min per modello. La cella 25 stimava circa 400 ms per

iterazione (circa 40 min): la stima includeva il warm-up e le ricompilazioni dei kernel su MPS. Il tempo vero è circa 40 ms per iterazione.

4.9 Celle 36–37 – Adapter e rollout

```
def deterministic_next_velocity(model, trajectory, current_velocity):
    batch, n = _single_batch(trajectory, current_velocity)
    du = denormalise_du(model(node_features(batch, u_now), batch))
    return (u_now + du * dyn_mask[..., None])[0, :n]       # physical units

def generative_next_velocity(model, trajectory, current_velocity, generator):
    batch, n = _single_batch(trajectory, current_velocity)
    du = denormalise_du(flow_matching_sample(model, batch, FM_EULER_STEPS, generator))

    return (u_now + du * dyn_mask[..., None])[0, :n]
```

- Gli adapter restituiscono il campo *completo* in unità fisiche, forma $(N, 2)$. Si de-normalizza dentro l'adapter. L'incremento si somma solo ai nodi dinamici; poi `enforce_known_boundaries` (dato) ripristina inflow e wall.
- Il rumore del FM usa il `generator` passato dal valutatore (seed $1234 + 1000 \cdot \text{traj} + \text{sample}$), così 5 rollout diversi per traiettoria, riproducibili.
- Rollout: parte dallo stato vero a $t = 0$ e fa 6 passi da 1 s. L'output di ogni passo diventa l'input del successivo (autoregressivo).
- Piccola inefficienza: `_single_batch` ricostruisce il grafo (`np.unique` sugli archi) a ogni passo da 1 s. Si potrebbe fare una volta per traiettoria (spunto per la 7b).

5 Parte 1 – Risultati e come interpretarli

5.1 Curve di loss

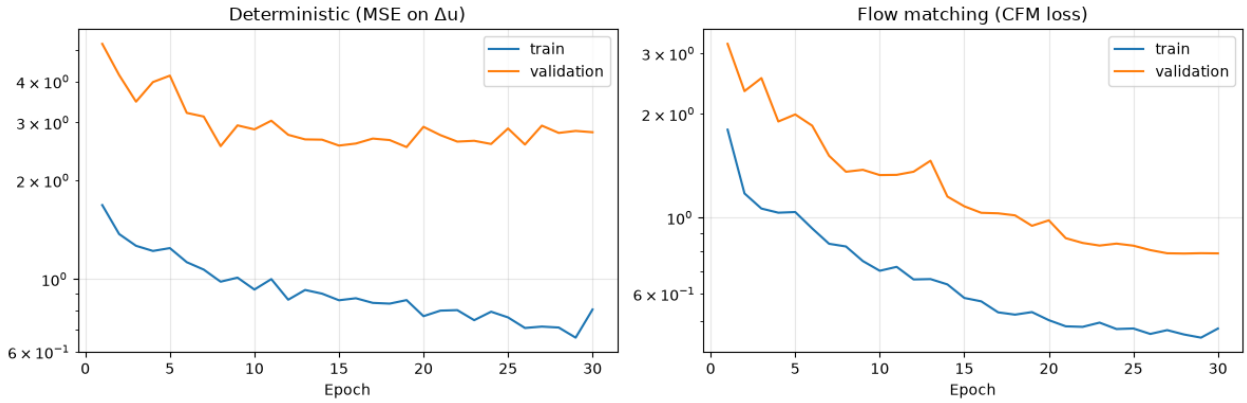


Figure 2: Loss di training e validation (scala log). Sinistra: MSE su $\Delta\mathbf{u}$ normalizzato. Destra: CFM loss.

	Deterministico (MSE)		Flow matching (CFM)	
epoca	train	val	train	val
1	1.681	5.208	1.800	3.196
3	1.263	3.472	1.062	2.542
8	0.982	2.541	0.824	1.358
14	0.903	2.659	0.639	1.151
19	0.863	**2.525** (best)	0.531	0.946
20	0.771	2.910	0.503	0.981
25	0.764	2.877	0.476	0.828
28	0.712	2.788	0.457	**0.786** (best)
30	0.808	2.802	0.476	0.787

Deterministico. Il training scende in modo regolare (minimo 0.663 all'epoca 29). La validation scende fino all'epoca 8 (2.54) e poi oscilla tra 2.5 e 2.9 senza migliorare: il gap con il training cresce, che è il segnale di overfitting descritto in 5c. Le oscillazioni vengono da (i) epoche corte (200 batch casuali) e (ii) validation su sole 20 traiettorie.

Flow matching. Entrambe le curve scendono in modo quasi monotono e si appiattiscono negli ultimi 5 epoch (anche perché il cosine schedule porta il learning rate a zero). Il rapporto val/train finale è circa 1.65 e costante, senza risalite: nessun overfitting evidente.

Punto debole / trappola: i due numeri da saper difendere

(1) Il deterministico batte il predittore banale? I target sono normalizzati per componente (media 0, varianza 1 sul training) e la loss somma 2 componenti. Quindi predire sempre l'incremento medio dà MSE ≈ 2 sul training; predire "nessun cambiamento" ($\Delta\mathbf{u} = 0$, persistenza) dà praticamente lo stesso valore, perché `du_mean` è quasi zero. La miglior validation è 2.52. Due letture possibili, da dire con onestà:

- le 20 traiettorie di validation hanno incrementi più grandi (regimi più attivi) di quelli medi del training, quindi anche la baseline banale lì è sopra 2: non l'abbiamo calcolata;
- oppure, a 1 s, la media condizionata ha davvero poco potere predittivo *puntuale*: proprio il punto dell'assignment.

Se avete 5 minuti prima di venerdì, calcolate la baseline sulla validation (snippet sotto): vi dà un numero concreto da citare.

(2) Gap già all'epoca 1. Con l'overfitting il gap cresce *durante* il training; qui c'è già all'inizio (1.68 vs 5.21). Oltre all'overfitting, quindi, c'è una differenza tra le distribuzioni di train e validation (pochi regimi di flusso diversi nelle 20 traiettorie). Il training loss di un'epoca è inoltre una media *durante* l'epoca, mentre la validation è calcolata alla fine.

```
@torch.no_grad()
def trivial_baselines():
    """MSE (normalised du, dynamic nodes) of two trivial predictors on the fixed
    validation pairs."""
    persist, mean_inc = [], []
    for b in valid_batches:
        b = to_device(b)
        target = normalise_du(b["u_next"] - b["u_now"])
        zero_du = normalise_du(torch.zeros_like(b["u_now"]))      # persistence: u(t
    +1) = u(t)
        persist.append(masked_mse(zero_du, target, b["dyn_mask"]).item())
        mean_inc.append(masked_mse(torch.zeros_like(target), target, b["dyn_mask"]).
    item())
    return np.mean(persist), np.mean(mean_inc)
print(trivial_baselines())   # compare with best deterministic validation loss 2.525
```

5.2 Wake temporal activity (test, 20 traiettorie)

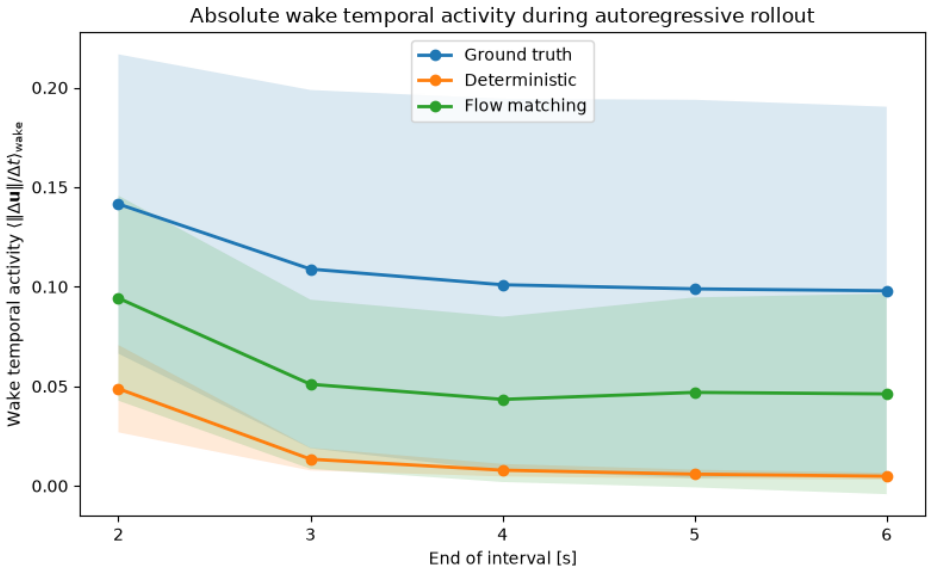


Figure 3: Wake temporal activity durante il rollout: media su 20 traiettorie di test, banda ± 1 deviazione standard tra traiettorie.

	Ground truth			Deterministico			Flow matching		
intervallo	media	std	mediana	media	std	mediana	media	std	mediana
$1 \to 2$	.1415	.0751	.1491	.0488	.0219	.0452	.0942	.0515	.0867
$2 \to 3$	.1088	.0899	.0864	.0134	.0057	.0124	.0510	.0424	.0384
$3 \to 4$	.1010	.0934	.0714	.0078	.0031	.0074	.0434	.0415	.0310
$4 \to 5$	.0989	.0949	.0737	.0059	.0022	.0055	.0469	.0477	.0206
$5 \to 6$	.0979	.0924	.0798	.0048	.0018	.0044	.0462	.0503	.0272

Letture principali.

1. **Ground truth:** parte da 0.14 e si stabilizza intorno a 0.10. Il primo intervallo risente ancora del transitorio iniziale. Da $2 \rightarrow 3$ in poi la scia è statisticamente stazionaria, quindi l'attività è circa costante.
2. **Deterministico:** $0.049 \rightarrow 0.005$, cioè un decadimento di un fattore 10. Ogni passo predice la media condizionata (D1). Il rollout converge verso un *punto fisso* vicino al campo medio nel tempo, dove $\Delta\mathbf{u} \approx 0$. La deviazione standard tra traiettorie è minuscola: tutte le traiettorie collassano nello stesso modo.
3. **Flow matching:** parte da 0.094 e si stabilizza a circa 0.045, cioè non decade ma resta a circa metà del ground truth.
4. **Media vs mediana (osservazione in più rispetto ad ANS):** per FM, dopo il primo intervallo, la mediana (0.02–0.04) è molto sotto la media (≈ 0.045). La distribuzione è asimmetrica: una parte delle traiettorie e dei campioni mantiene una scia attiva, molti altri diventano quasi statici. Coerente con le GIF ("some samples go static"). Per il GT la mediana (0.07–0.09) è più vicina alla media.
5. **Bande enormi per il GT:** la deviazione standard (≈ 0.09) è quasi uguale alla media. È fisica, non rumore del modello: alcune traiettorie hanno Reynolds basso (scia quasi stazionaria, attività vicina a zero), altre hanno shedding forte. La banda $\pm 1\sigma$ va anche sotto zero per costruzione (std simmetrica su una quantità positiva).

5.3 Rollout visivi

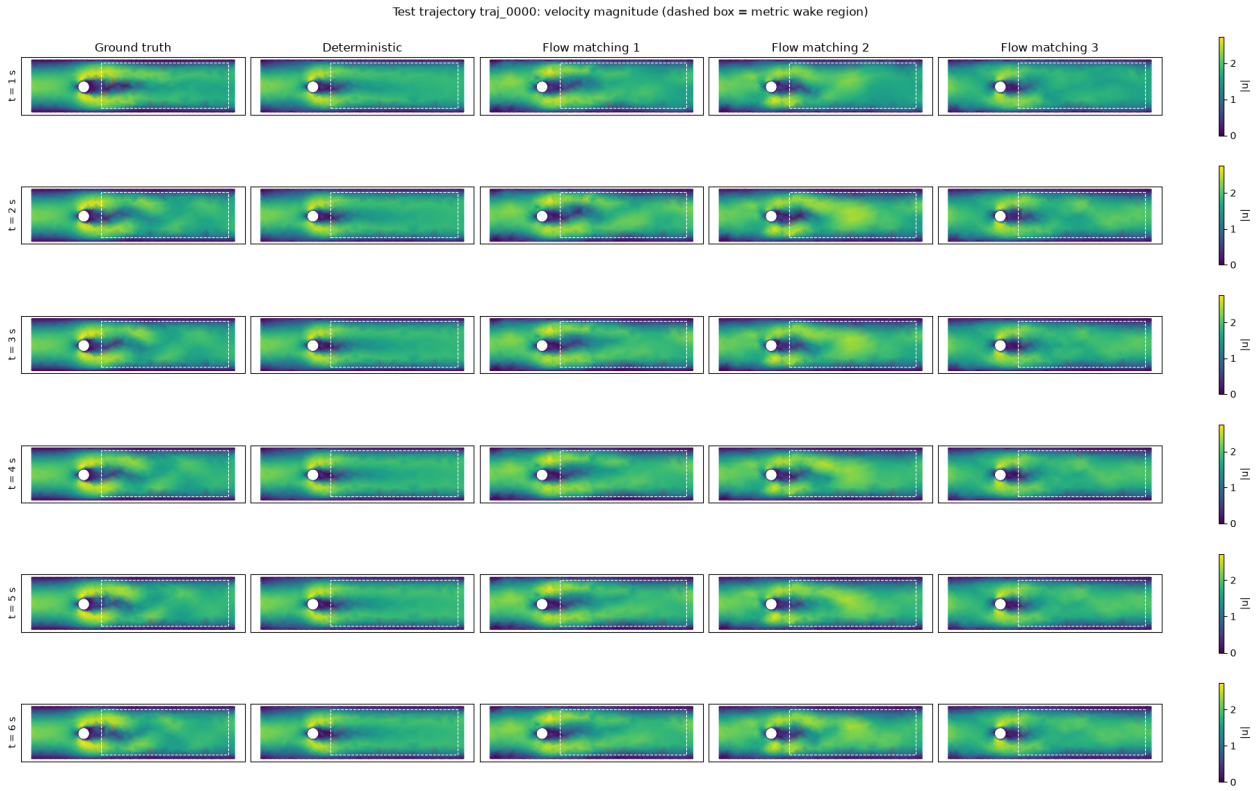


Figure 4: Modulo della velocità, test traj_0000, colonne GT / deterministico / 3 campioni FM.

Teoria ↔ pratica: perché FM resta sotto il ground truth: ipotesi ordinate per plausibilità

1. **Exposure bias / accumulo di errore:** il modello è addestrato solo su input veri (teacher forcing). Dopo qualche passo l'input è un campo leggermente più liscio del vero; per input lisci la distribuzione condizionata appresa ha poca attività. Lo stato "statico" agisce

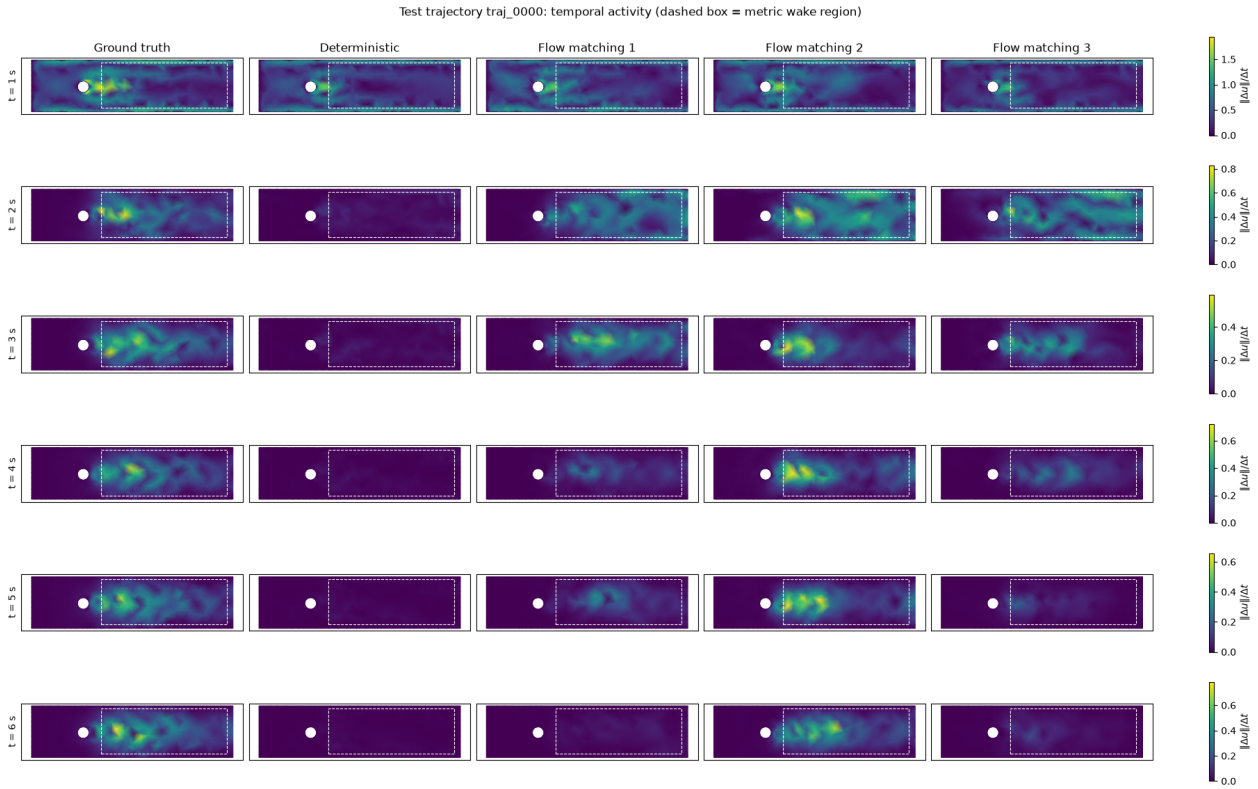


Figure 5: Attività $\|\mathbf{u}(t) - \mathbf{u}(t - 1\text{s})\|$ per nodo (la quantità che la metrica media). Il deterministico è nero dopo $t = 2$ s. FM 2 mantiene la scia attiva fino a 6 s, FM 1 e FM 3 si spengono gradualmente. A $t = 1$ s il GT cambia ovunque, anche a monte: è il transitorio (burn-in).

quindi da attrattore. Rimedi: rumore sugli input in training (come MGN), push-forward.

2. **Sotto-addestramento:** solo 6000 step (circa 4 minuti); la loss FM stava ancora scendendo lentamente e il modello ha 0.2 M parametri.
3. **Errore di integrazione:** 20 passi di Euler tagliano le traiettorie curve verso la media (D3), cioè meno dettagli fini.
4. **Condizionamento su τ debole:** τ entra solo come canale di input al primo layer, e deve "sopravvivere" attraverso 4 blocchi.
5. **Rumore i.i.d. per nodo:** sulla mesh non uniforme il rumore non ha correlazione spaziale. Più nodi vicino al cilindro significa più gradi di libertà di rumore lì. Il modello deve imparare a trasformare rumore bianco "a densità variabile" in strutture coerenti.

5.4 Lo sweep sui passi di Euler (cella 41): cosa è successo

La cella è stata scritta per tre analisi su **validation**: (1) MSE di un singolo campione, MSE della media di più campioni, spread tra campioni, MSE del deterministico, per $K \in \{1, 2, 5, 10, 20, 50\}$; (2) attività di scia media per K; (3) snapshot con lo stesso seed. **Non ha prodotto output:** `RuntimeError: MPS backend out of memory`. In più `rollout_activity_validation` non è definita da nessuna parte.

Da sistemare prima della consegna: se volete numeri veri per la 5e

Rilanciate solo la parte (1), con meno batch e campioni e liberando memoria prima:

```
import gc; gc.collect(); torch.mps.empty_cache()
display(one_step_vs_steps(steps_list=[1, 2, 5, 10, 20], n_samples=3, n_batches=4))
```

Cosa aspettarsi in teoria (scrivetelo come ipotesi, poi confrontatelo con i numeri):

- $K = 1$: spread ≈ 0 (tutti i campioni uguali) e MSE $\approx$ MSE del deterministico (D3);
- al crescere di K: lo spread cresce e si satura; l'MSE del singolo campione *cresce* (un campione nitido con fase sbagliata paga di più); l'MSE della media dei campioni resta vicino a quello del deterministico;
- relazione utile: $\mathbb{E}\|\hat{x} - y\|^2 \approx \|\bar{x} - y\|^2 + \text{spread}$ (MSE di un campione = MSE della media + varianza tra campioni).

Da dire all'interview (EN): "what did you observe when changing the number of steps?"

"Honestly, our sweep ran out of GPU memory on the laptop, so the answer in ANS is based on the theory, not on our measurements. What we expect is this: with one Euler step from pure noise, the update is $x_0 + v(x_0, 0)$, and the optimal $v(x, 0)$ is $\mathbb{E}[x_1 \mid C] - x$, so every sample becomes the conditional mean, exactly the deterministic prediction. With more steps the samples can follow the curved marginal trajectories and keep their diversity, until the result saturates. Below roughly ten steps we would expect smoother and less diverse samples, and therefore lower wake activity."

6 Parte 1 – Scelte fatte, alternative non fatte, “what if”

6.1 Tabella delle scelte

Aspetto	Scelta vostra	Perché	Alternative e trade-off
Rappresentazione	grafo dei lati dei triangoli, bidirezionale	segue la mesh non uniforme, gestisce N e connettività variabili, niente interpolazione	griglia + CNN/U-Net (interpolazione, perdita di risoluzione vicino al cilindro, maschere per la geometria); grafo radius/kNN sui nodi (ignora la connettività vera)
Processor	4 blocchi: 1 message passing + 1 attenzione globale	locale e geometrico + campo recettivo globale per il salto di 1 s; mesh piccole ($\approx$ 600 nodi)	MGN puro con molti step (costoso, campo recettivo lento); **MGN multiscala** (coarsening/ pooling su grafi più grossolani, U-Net su grafo); **nodo virtuale** globale; transformer puro (perde la connettività); **Transolver** (attenzione su “slice” fisiche, lineare in N, nella letteratura del corso)
Aggregazione	somma	come MGN; gradi uniformi nella mesh; conserva il numero di vicini	media (parte 2), max, attenzione sui vicini
Target	$\Delta\mathbf{u}$ normalizzato	scala unitaria; residuo $\approx$ passo di Euler; il modello non deve ricopiare il campo	u_{t+1} diretto (più facile da far “derivare”), $\Delta\mathbf{u}/\Delta t$, più passi insieme (time bundling)
Normalizzazione	z-score per componente (u, $\Delta\mathbf{u}$); posizione in $[-1, 1]$; archi / lunghezza media	solo dal train; componenti con scale molto diverse ($u_y \ll u_x$)	nessuna; per-nodo; normalizzazione robusta (mediana/IQR)
Feature di nodo	$\tilde{u}$, posizione assoluta, one-hot tipo	condizione completa; posizione per l’attenzione e per pareti/scia	distanza dal cilindro, posizione relativa al centro del cilindro, Fourier features, proxy di Reynolds (raggio, velocità d’inflow) come feature esplicite
Embedding di τ	canale scalare concatenato all’input	semplice; come la U-Net del Practical 3	embedding sinusoidale + MLP (parte 2); **adaLN/FiLM** in ogni blocco (come i diffusion transformer): τ influenza ogni layer
Condizionamento	$\mathcal{C}_t$ come feature di input, identico al deterministico	confronto pulito; tutta la rete vede la condizione	encoder separato + cross-attention; adaLN; cambiare il prior (vedi sotto)
Prior	$\mathcal{N}(0, I)$ i.i.d. per nodo, zero sui nodi di bordo	standard, richiesto dalla formulazione	**prior informato**: partire dalla predizione deterministica + rumore (“residual” FM), così il trasporto modella solo l’incertezza; rumore correlato spazialmente o scalato con l’area locale
Percorso	rettilineo, $x_\tau = (1-\tau)x_0 + \tau x_1$	quello della lezione e del testo	percorsi VP/cosine (diffusion); coupling a trasporto ottimo per minibatch

Aspetto	Scelta vostra	Perché	Alternative e trade-off
Campionamento di τ	uniforme	standard	logit-normale (più peso a τ intermedi)
Solver	Euler esplicito, 20 passi	semplice, 20 chiamate per secondo fisico	Heun/midpoint (2 chiamate per passo, ordine 2): meno passi totali; RK4; solver adattivo
Schema di training	teacher forcing, coppie a un passo	semplice, parallelo	rumore sugli input (MGN), **push-forward**, loss su rollout di più passi
Peso della loss	media sui nodi dinamici	è la loss del testo	**peso per area**, come nella metrica: oggi le zone fitte vicino al cilindro contano di più nella loss che nella metrica
Model selection	best validation loss a un passo	semplice ed economica	metrica di rollout (wake activity) su validation: la CFM loss non misura la qualità dei campioni
Batching	padding a 2 forme fisse	attenzione densa; niente ricompilazioni su MPS	grafo disgiunto (PyG) + attenzione con maschera a blocchi
Budget	6000 step, batch 8, identico per i due modelli	confronto equo; vincoli di tempo	training più lungo (la FM non aveva finito di convergere)
Data augmentation	nessuna	–	riflessione rispetto all'asse del canale ($y \mapsto 0.41 - y$, $u_y \mapsto -u_y$): NS è equivariante per riflessione e il cilindro riflesso è solo un'altra geometria valida (assumendo un profilo d'inflow simmetrico)

6.2 "How would your solution change if . . . "

Il docente ha scritto esplicitamente che può chiedere come cambierebbe la soluzione se cambiassero i requisiti. Ecco i casi più probabili, con la risposta in inglese.

Da dire all'interview (EN): . . . the prediction horizon were 0.01 s instead of 1 s?

"Then the map would be nearly deterministic: in 0.01 s the vortex phase hardly changes, so the conditional distribution is very narrow and the deterministic model would be about as good as flow matching, at a twentieth of the cost (that is our answer 3b). Information would only travel a few mesh cells per step, so plain MeshGraphNets-style message passing would be enough and the global attention would matter less. The new problem would be long rollouts: 600 steps instead of 6, so error accumulation becomes the main issue, and we would add input noise or push-forward training."

Da dire all'interview (EN): . . . the meshes had 100,000 nodes, or were 3D?

"Full attention is quadratic in the number of nodes, so it would no longer fit. We would keep local message passing and get the global receptive field another way: a multiscale or hierarchical graph network that coarsens the mesh, a few virtual global nodes, or linear-cost attention such as Transolver's physics attention, which attends over a small number of learned slices instead of all nodes. The flow-matching part would stay the same."

Da dire all’interview (EN): . . . the geometry changed, e.g. an airfoil instead of a cylinder?

“The graph representation handles any mesh, so the architecture would still work. But we use absolute positions normalised with the coordinate range of this channel, so the model has implicitly learned this domain and would need retraining. To generalise across geometries we would rely more on geometry-relative features, such as the distance to the nearest wall or object surface, and less on absolute coordinates.”

Da dire all’interview (EN): . . . you only had 50 training trajectories?

“We would add inductive bias and reduce capacity: drop the absolute positions so that message passing is translation invariant, make the model smaller, and use data augmentation such as the reflection about the channel centre line. Attention with absolute positions has to learn locality from data, which needs more data.”

Da dire all’interview (EN): . . . you also had to predict the pressure?

“Add pressure as an extra input and output channel with its own normalisation. Pressure in incompressible flow is determined globally by an elliptic equation, so a global receptive field would be even more important. That is another argument for the attention layers.”

Da dire all’interview (EN): . . . sampling had to be much faster?

“The cost is the 20 network calls per physical second. Options: fewer steps with a second-order solver such as Heun; start from an informed prior, for example the deterministic prediction plus noise, so the flow only has to model the residual uncertainty and the paths are shorter; or straighten the flow (rectified flow or reflow) or distil it into a one or few-step model.”

Da dire all’interview (EN): . . . you wanted an uncertainty estimate, not just samples?

“Draw many samples from the same initial condition and compute statistics per node: mean, standard deviation, quantiles. The spread grows over the rollout where the dynamics are uncertain, mainly in the wake. The deterministic model cannot give this at all. Caveat: the spread is only meaningful if the model is calibrated, which we did not test.”

Da dire all’interview (EN): . . . the Reynolds number were given as an input?

“We would add it as a global conditioning variable, either as an extra node feature or, better, through FiLM or adaptive layer norm. Right now the model has to infer the regime from the inflow velocity and the cylinder size, which it can do, but an explicit input would make it easier.”

Da dire all’interview (EN): . . . the inflow were noisy or time-varying?

“Then there would be genuine aleatoric uncertainty, not only uncertainty from lost information, and a probabilistic model would be even more justified. The boundary values would no longer be constant, so we could not simply restore them. We would feed the future boundary condition as an additional input, or model it as part of the state.”

7 Parte 1 – Le vostre risposte ANS, commentate

Per ogni domanda: sintesi di cosa avete scritto, cosa regge bene, dove il docente può insistere e le risposte pronte ai follow-up.

7.1 2a–2c: CNN, Transformer, GNN

Cosa avete scritto. CNN: solo parzialmente adatta. Serve una griglia, quindi interpolazione avanti e indietro a ogni passo (risoluzione sprecata o insufficiente, geometria con maschere, errore che si accumula, connettività persa); è locale, quindi servirebbe una U-Net profonda. Il message passing generalizza la convoluzione a vicinati irregolari. Transformer: sì, con caveat. Gestisce insiemi di dimensione variabile, ha campo globale e costa poco con circa 600 nodi, ma non ha nozione di spazio e perde connettività e geometria relativa. Ne tenete la parte globale. GNN: sì, è la spina dorsale. La mesh è un grafo; MGN; il limite è il campo recettivo, risolto con l'attenzione (stile GraphGPS).

Valutazione. Risposte solide e coerenti con il codice. Il collegamento "message passing = convoluzione su vicinati irregolari, con località e weight sharing" è esattamente il linguaggio della lezione 02.

Da dire all'interview (EN): follow-up: "in what sense is message passing a generalisation of convolution?"

"A convolution applies the same kernel at every pixel to a fixed neighbourhood, which gives locality, weight sharing and translation equivariance. Message passing applies the same message and update functions at every node to its graph neighbourhood. That keeps locality and weight sharing, but the neighbourhood can have any size and shape, and the aggregation is a sum, so the result does not depend on the order of the neighbours: it is permutation equivariant. On a regular grid, with relative positions as edge features, message passing can represent a convolution."

Da dire all'interview (EN): follow-up: "would a U-Net on an interpolated grid really be worse?"

"Not necessarily worse in accuracy if the grid were fine enough, and MeshGraphNets compares exactly against that baseline. But to resolve the boundary layer around the cylinder a uniform grid must be fine everywhere, which is wasteful. The cylinder changes position and size, so we would need masks, and interpolating back and forth at every rollout step adds error that accumulates. The graph avoids all three problems."

Punto debole / trappola

In 2b dite "node positions provided as input features" come sostituto del positional encoding. È corretto ma grezzo: due coordinate grezze sono un encoding a bassa frequenza, e l'attenzione fatica a distinguere nodi vicini. Alternative: Fourier features delle coordinate, oppure posizione relativa come bias dell'attenzione. Se vi chiedono "how does attention know where nodes are?": tramite le coordinate nelle feature di nodo e, indirettamente, tramite i latenti già arricchiti dal message passing, che conoscono il vicinato geometrico.

7.2 3a–3b: perché un modello probabilistico

3a. Dinamica deterministica, ma osservata in modo discreto: informazione persa tra i nodi, stati diversi uguali sulla mesh. Sistema sensibile alle condizioni iniziali, quindi lo stesso input ha molti futuri possibili. L'MSE impara la media condizionata, che liscia i vortici; il modello probabilistico dà campioni nitidi e una stima dell'incertezza.

3b. Il deterministico va bene per orizzonti brevi, dove ogni stato dipende quasi solo dal precedente e piccoli errori non si amplificano, e per sistemi ben risolti dalla discretizzazione. In quel caso la media condizionata è il vero esito, e il deterministico costa una sola chiamata invece di molte.

Punto debole / trappola

- "chaotic": vedi box 3.2, preparate la versione sfumata (fase neutralmente stabile).
- 3b può essere rafforzata con la formula: il deterministico è ottimo quando $\mathrm{Cov}(\Delta\mathbf{u} \mid \mathcal{C}_t) \approx 0$, cioè quando la distribuzione condizionata è quasi un delta. Allora $\mathbb{E}[\Delta\mathbf{u} \mid \mathcal{C}_t]$ *è* un campione tipico. Esempi: diffusione/calore, flussi laminari stazionari a basso Reynolds, passi molto brevi. Notate che nel vostro dataset ci sono traiettorie quasi stazionarie (Reynolds basso) dove il deterministico è probabilmente quasi perfetto.

Da dire all'interview (EN): follow-up: "what is the difference between aleatoric and epistemic uncertainty here?"

"Aleatoric uncertainty is randomness in the system itself, which more data cannot remove. Epistemic uncertainty comes from what we do not know, for example the information lost between mesh nodes or the model's own errors. Here the physics is deterministic, so the spread mostly comes from partial observation of the state and sensitivity of the vortex phase. From the model's point of view it behaves like aleatoric noise, because the input simply does not determine the output exactly."

7.3 4a: architettura del deterministico

Valutazione. Risposta molto completa e fedele al codice: rappresentazione (lati bidirezionali, deduplicazione, padding con maschere), 8 feature di nodo e 3 d'arco, blocco MP + attenzione, decoder, de-normalizzazione, iperparametri. I numeri coincidono con il notebook (197,506 parametri).

Possibili domande di dettaglio, con risposte brevi:

- **Why sum aggregation?** "As in MeshGraphNets. Node degrees in a triangle mesh are fairly uniform, about six, and the sum keeps the information about how many neighbours there are. A mean would lose it."
- **Why residual connections and LayerNorm?** "They keep training stable with several blocks and a relatively high learning rate, and the residual form lets each block learn a correction."
- **Why update edges too?** "Edge latents carry the relative geometry through the blocks. Updating them, as in MeshGraphNets, lets the network build richer pairwise information than the three raw edge features."
- **Why only 4 blocks?** "Message passing alone could not reach far enough in any reasonable number of steps, and attention gives the global range anyway. Four blocks were a compromise with training time on a laptop. We did not tune this on the test set."
- **How do padded nodes not leak?** "Padded edges have their messages multiplied by zero, padded nodes are excluded from attention through the key padding mask, and they have zero weight in the loss."

Punto debole / trappola

"translation-invariant" (vedi box nella sez. 4): vale per gli archi, non per il modello intero. Preparate la risposta.

7.4 4b: embedding del tempo di flusso τ

Cosa avete scritto. τ scalare, uno per campione, broadcast a tutti i nodi e concatenato come canale in più, insieme a x_τ (2 canali): $8 + 2 + 1 = 11$ canali. Come la U-Net di Navier–Stokes del Practical 3. Integrazione con 20 passi di Euler.

Punto debole / trappola: follow-up probabile: "is concatenation at the input enough?"

È la scelta più semplice. Il suo limite: τ entra solo nel node encoder e deve sopravvivere attraverso 4 blocchi. Il comportamento corretto cambia molto tra $\tau \approx 0$ (input quasi solo rumore, la rete deve puntare verso la media condizionata) e $\tau \approx 1$ (input quasi pulito, piccole correzioni). Alternative: embedding sinusoidale + MLP (lo avete fatto nella parte 2), e soprattutto FiLM/adaLN in ogni blocco.

Da dire all'interview (EN)

"We concatenate τ as an input channel, like the U-Net in Practical 3. It works, but the information only enters at the first layer. A stronger option is a sinusoidal embedding of τ followed by an MLP that predicts a scale and shift for the layer norms in every block, as in diffusion transformers. In part 2 we used a sinusoidal embedding added to every token, which is in between. With more time we would try adaptive layer norm in part 1."

7.5 4c: condizionamento e differenze tra i due modelli

Cosa avete scritto. La condizione $\mathcal{C}_t$ (velocità attuale, geometria, tipi) entra come nel deterministico, come feature, ed è fissa durante l'ODE. Alternative: (1) adaLN/FiLM con condizione e τ; (2) encoder separato + cross-attention; (3) cambiare la distribuzione sorgente (bridge / stochastic interpolant che parte vicino allo stato attuale). Differenze: 3 canali in più (+192 parametri), output = campo di velocità in spazio $\Delta\mathbf{u}$ normalizzato, loss CFM con rumore e τ nuovi a ogni step, rumore e target nulli sui bordi, 20 chiamate per passo, campione diverso a ogni chiamata.

Valutazione. Ottima. L'opzione (3) è la più interessante concettualmente, ed è proprio quello che avete fatto nella parte 2 (prior informato). È un bel collegamento da fare all'interview.

Da dire all'interview (EN): collegamento tra le due parti

"The third option is essentially what we did in part 2 with the informed prior: instead of always starting from pure noise, the source distribution already depends on the condition, so the flow only has to transport a guess to the target. In part 1 a natural version would be to start from the deterministic prediction plus noise, so the flow models only the uncertainty around the conditional mean."

7.6 5a–5b: procedura di training

Valutazione. Corretta nel contenuto, ma **correggere l'hardware** (MPS, non CUDA) e **aggiungere il tempo** in 5a (circa 8 s per epoca, circa 4 minuti in totale).

Da dire all'interview (EN): "why AdamW with cosine decay and clipping?"

"AdamW is a robust default for networks with attention and LayerNorm, with decoupled weight decay as mild regularisation. The cosine schedule decays the learning rate smoothly to zero, which helps the final convergence, and it is what Practical 3 used. Gradient clipping protects against occasional large gradients, for example from batches with large meshes or, for flow matching, from samples with very small or very large τ."

Da dire all'interview (EN): "why random pairs instead of full epochs?"

"There are 250,000 pairs, but consecutive pairs 0.01 s apart are almost identical, so a full pass would be very redundant. Sampling random trajectories and random start times gives diverse batches, and loading everything into RAM avoids slow reads from the compressed HDF5 file."

7.7 5c–5d: curve di loss

Valutazione. Interpretazioni corrette: overfitting e plateau per il deterministico (con parte irriducibile dell'errore), convergenza regolare per FM, CFM loss non confrontabile e mai a zero. Vedi sez. 5.1 per i due punti scomodi (baseline ≈ 2 e gap già all'epoca 1).

Da dire all'interview (EN): "why can the flow-matching loss never reach zero?"

"Because the regression target is $x_1 - x_0$, and for a given noisy point x_τ there are many pairs (x_0, x_1) that pass through it. The best the network can do is predict the conditional average direction $\mathbb{E}[x_1 - x_0 \mid x_\tau, \tau]$, and the remaining variance around that average is irreducible. So even a perfect model has a positive loss, and its value cannot be compared with the deterministic MSE."

Da dire all'interview (EN): "your validation loss for flow matching is lower than for the deterministic model; is FM better?"

"No, the two numbers measure different things. The deterministic loss is the error on the velocity increment; the flow-matching loss is the error on a vector field in noise-to-data space. They are not on the same scale, and a lower flow-matching loss does not even guarantee good samples. The comparison has to be made on the rollouts, with the wake activity and the visualisations."

7.8 5e: numero di passi di integrazione

Cosa avete scritto. Meno passi: costo minore, ensemble più grandi a parità di budget. Ma Euler assume velocità costante sul passo: sulle traiettorie curve si esce dal percorso e gli errori si accumulano. Vantaggi: velocità, latenza. Svantaggi: mode averaging e blur (al limite di un passo, regressione diretta), accumulo nel rollout.

Da sistemare prima della consegna

Sostituire "every sample collapses to the data mean" con "every sample collapses to the *conditional* mean $\mathbb{E}[x_1 \mid C_t]$, i.e. the deterministic prediction". Aggiungere una frase onesta: "Our sweep over the number of steps ran out of memory, so this answer is based on theory." Oppure rilanciate lo sweep ridotto (sez. 5.4).

Da dire all'interview (EN): "why are the trajectories curved if you trained on straight lines?"

"Each conditional path from one noise sample to one data sample is a straight line. But the network learns the average of these directions at each point, and that average changes as τ increases: near $\tau = 0$ the point could still go to many targets, near $\tau = 1$ it is committed to one. So the marginal trajectories bend, and a coarse Euler step cuts the corner. In the limit of one step the sample lands exactly on the conditional mean."

7.9 6b, 6d, 6e: GIF qualitative

Valutazione. Buone e coerenti con le figure: deterministico statico dopo il primo stato, struttura media corretta (alta velocità sopra e sotto il cilindro, zona lenta dietro e vicino alle pareti), scia

liscia. FM: "wisps" che si staccano dal cilindro, qualità che dipende dal campione, alcuni campioni diventano statici, fase non uguale al ground truth (atteso).

Da dire all'interview (EN): "if a FM sample does not match the ground truth, why is it good?"

"Because the goal is not to reproduce the one true future, which is not predictable after a few shedding periods, but to produce futures that are physically plausible and statistically like the real one. A sample with vortices at a slightly different phase is a valid future. The deterministic model matches the ground truth better on average, but it does so by producing a field that never occurs in reality."

7.10 6g: analisi della metrica

Valutazione. Corretta e coerente con la tabella. Si può arricchire con due osservazioni che il docente apprezzerebbe:

1. la **mediana** FM è molto sotto la media (distribuzione asimmetrica: parecchi rollout si spengono, pochi restano molto attivi);
2. la grande deviazione standard del GT riflette la **varietà di regimi** tra le traiettorie, non rumore.

Da dire all'interview (EN): "why does the deterministic activity decay instead of staying constant at a lower level?"

"Each step predicts the conditional mean, which is smoother than any real state. The next step takes this smoothed state as input and averages again. Repeated averaging drives the rollout towards a fixed point close to the time-averaged flow, where the predicted increment is almost zero. That is why the activity decays by a factor of ten instead of staying at a lower constant level."

7.11 7a–7b: bozze pronte (le risposte sono vuote in ANS)

Da sistemare prima della consegna: 7a – Non-obvious limitations (bozza, circa 170 parole)

(1) Both models are trained only on true input states (teacher forcing). In the rollout the input becomes the model's own, slightly smoother prediction, which it never saw during training; this exposure bias is a likely reason why some flow-matching samples gradually become static. Input noise or push-forward training would address it. (2) The loss is a mean over nodes, so the densely meshed region around the cylinder weighs more in training than in the area-weighted metric. (3) The flow time τ enters only as an input channel at the first layer, so its effect must survive four blocks. (4) The prior noise is i.i.d. per node, so on a non-uniform mesh its spatial structure depends on the mesh density. (5) Model selection uses the one-step validation loss, which for flow matching does not measure sample quality. (6) Absolute positions are normalised with this channel's coordinates, so the model is tied to this domain. (7) Training was short (6,000 steps), and the flow-matching loss had not fully converged.

Da sistemare prima della consegna: 7b – Changes with most impact on efficiency (bozza, circa 150 parole)

The dominant cost is sampling: each 1-second step needs 20 network evaluations, and the evaluation runs 5 rollouts of 6 steps for 20 trajectories. (1) Use a second-order solver (Heun or midpoint) with fewer steps, which needs fewer evaluations for similar accuracy. (2) Start from an informed prior, the deterministic prediction plus noise, so the flow only models the residual uncertainty, paths are shorter and straighter, and fewer steps are needed. (3) Outside the supplied evaluator, which requests one sample at a time, generate the samples of a trajectory in one batch. (4) Cache the graph of each trajectory instead of rebuilding the edge list at every rollout step. (5) Use mixed precision and the disjoint-graph batching, which avoids computation on padded nodes. (6) In the longer term, distil the flow into a model that needs one or a few steps.

8 Parte 2 – Pedoni a Eindhoven Centraal

Punto debole / trappola: fonte

Questa sezione si basa solo sul testo dell'assignment e sulle vostre risposte ANS: il notebook della parte 2 non era tra i file. I dettagli di implementazione (per esempio la forma esatta della crescita del rumore in V2) vanno verificati sul vostro codice. All'interview chi ha scritto la parte 2 deve saperli a memoria.

8.1 Il problema in breve

- 100,000 scene, ognuna un array $(N, 30, 2)$ con $N \in [5, 100]$ pedoni, 30 frame a $\Delta t = 0.5$ s (15 s), posizioni in metri. Pickle da circa 287 MB, lista di array (non impilabili perché N varia). Ogni pedone è presente in tutti i 30 frame.
- Condizione: frame 0–9 (posizioni, più derivate). Target: velocità dei frame 10–29, $x_1 = (v^{10}, \dots, v^{29}) \in \mathbb{R}^{N\times 20\times 2}$, generate **tutte insieme** (time bundling). Posizioni ricostruite con $p^{9+k} = p^9 + \Delta t \sum_{j=1}^{k} v^{9+j}$.
- Due modelli CFM identici tranne il prior: (I) gaussiano non informato $\pi = p_0(x_0)\, p_1(x_1 \mid c)$, $p_0 = \mathcal{N}(0, \sigma^2 I)$; (II) informato $\pi = p_0(x_0 \mid c)\, p_1(x_1 \mid c)$ con $x_0 \perp x_1 \mid c$, costruito solo dai frame osservati, stocastico, campionato allo stesso modo in training e inferenza.
- Valutazione: $M = 20$ futuri per scena; ADE, FDE, $\min_{20}$ADE, $\min_{20}$FDE (il minimo a livello di *scena*, non di pedone); istogrammi di velocità, accelerazione, tortuosità e distanza dal vicino più vicino (asse 0–2 m).

8.2 La vostra soluzione (dalle risposte ANS)

Dati	primi 10,000 campioni, split 7,000 / 1,000 / 2,000 (seed 42)
Grafo	un grafo per scena, kNN con $k = 8$ (o $N - 1$ se $N < 9$) calcolato sulle posizioni del frame 9; stessi archi per tutti i 30 frame
Feature	velocità (zero al frame 0), posizioni, entrambe normalizzate con statistiche di train; posizioni future ricostruite dalle velocità rumorose correnti; posizione relativa $x_j(k) - x_i(k)$ nei messaggi
Input della rete	sequenza di 30 frame per pedone: 10 osservati puliti + 20 futuri rumorosi; per ogni frame (velocità, posizione) $\to$ proiezione lineare a 128
Embedding aggiunti	frame index (appreso), tipo osservato/generato (appreso), τ (sinusoidale $\to$ MLP), sommati a ogni token
Blocco ($\times 4$)	(1) self-attention temporale, 4 teste, sui 30 frame di ciascun pedone; (2) message passing spaziale per frame, messaggio MLP$(h_i, h_j, x_j - x_i)$, **media** sugli 8 vicini, MLP di update; (3) feed-forward. Ognuno con LayerNorm e residual
Output	lineare sugli stati dei 20 frame futuri $\to (N, 20, 2)$ = campo di velocità del flow
Prior	Gaussiano $\sigma = 1$; V1: ultima velocità osservata + rumore $\propto$ std delle velocità osservate (minimo 0.1); V2: media delle ultime 3 velocità + rumore che cresce con l'orizzonte (growth 1.0, minimo 0.1)
Training	CFM, AdamW lr $3 \cdot 10^{-4}$, wd 0.01, batch 16 scene, 20 epoche $\times$ 438 step, clipping 1.0, modello dell'ultima epoca, circa 1.2 M parametri, seed 42
Inferenza	20 passi di Euler, 20 futuri per scena; T4: 1.47 s per minibatch di 32 scene per un futuro (circa 73 ms per chiamata), 29.4 s per 20 futuri

Teoria $\leftrightarrow$ pratica: spazio e tempo, lezione 03 slide 56–59

La lezione dice: struttura globale = insieme/grafo, quindi *equivarianza per permutazione*; asse temporale = griglia, quindi *equivarianza per traslazione nel tempo* (slide: GNN con encoder CNN su una finestra temporale). Voi avete fatto: **message passing** per lo spazio (permutation

equivariant) e **attenzione** per il tempo. Il frame encoding *appreso e assoluto* rompe l'equivarianza per traslazione temporale. Va bene, perché i ruoli dei frame sono fissi: 0–9 osservati e 10–29 da generare. Il frame 12 significa sempre "1.5 s dopo l'ultima osservazione". Una CNN 1D sarebbe l'alternativa "da slide".

8.3 Teoria del prior informato (11b): perché dovrebbe aiutare e perché V1 non ha aiutato

Teoria: cosa cambia con un prior che dipende da c

Con il coupling $\pi(x_0, x_1 \mid c) = p_0(x_0 \mid c)p_1(x_1 \mid c)$ il CFM resta valido: si interpola tra un campione del prior e il dato e si regredisce $x_1 - x_0$. Requisiti: $x_0 \perp x_1 \mid c$ (niente informazione dal futuro), prior stocastico, stessa procedura in training e inferenza. **Perché in teoria aiuta:**

- **distanza di trasporto più corta:** $\mathbb{E}\|x_1 - x_0\|^2$ è più piccolo, quindi la rete deve imparare un campo di velocità di modulo minore (è un "residual" rispetto a una buona ipotesi);
- **percorsi più dritti:** meno curvatura delle traiettorie marginali, quindi l'integrazione di Euler con pochi passi sbaglia meno (collegamento a D3);
- **incertezza strutturata:** il rumore può essere scalato per pedone e per orizzonte, mettendo la variabilità dove serve.

Teoria: perché V1 era troppo "stretto"

In V1 il rumore era proporzionale alla variabilità della velocità osservata, piccola per la maggior parte delle persone, e *costante su tutti i 20 frame*. Se il prior è molto concentrato, la rete deve *espandere* la massa di probabilità per generare futuri diversi: deve imparare un flusso con divergenza grande. Le reti tendono a dare mappe lisce, quindi i futuri restano simili tra loro (sotto-dispersione). I numeri lo confermano: ADE quasi uguale al gaussiano, FDE un po' migliore, ma $\min_{20}$ADE e $\min_{20}$FDE *peggiori*. Quando la persona vera fa qualcosa di diverso da "continua a velocità costante", nessuno dei 20 futuri le si avvicina.
V2 fa crescere il rumore con l'orizzonte. È esattamente l'intuizione della lezione 03: per un errore che si somma passo dopo passo come un random walk, RMSE $\approx \sigma\sqrt{t}$, cioè l'incertezza cresce con l'orizzonte. (Controllate nel codice se la crescita è lineare in k o $\propto \sqrt{k}$: sapere la forma esatta e saperla motivare è un punto facile.)

Da dire all'interview (EN): "your informed prior V1 was worse on minADE: why, and what did you learn?"

"The prior was too narrow. Its noise was proportional to how much the observed velocity varied, which is small for most people, and it was constant over all 20 future frames. So all 20 futures stayed close to 'keep walking at the same velocity'. The average error was similar to the Gaussian prior, but when the person actually did something different, none of the 20 samples covered it, so the best-of-20 metrics got worse. In V2 we made the noise grow with the prediction horizon, because uncertainty accumulates over time, a bit like a random walk, and we smoothed the guess by averaging the last three velocities. V2 is better on all four metrics."

8.4 Risultati (14a)

prior	ADE [m]	FDE [m]	$\min_{20}$ADE [m]	$\min_{20}$FDE [m]
Gaussiano	1.059	2.213	0.751	1.457
Informato V1	1.061	2.152	0.791	1.522
Informato V2 (growth 1.0)	**0.987**	**1.994**	**0.722**	**1.372**

- **ADE/FDE (media su 20 futuri)** premiano campioni tutti vicini al vero: favoriscono un modello poco disperso. **min**$_{20}$ premia la *copertura*: basta che uno dei 20 futuri sia vicino. Con dispersione zero, $\min_{20}\text{ADE} = \text{ADE}$. I due tipi di metrica vanno letti insieme: diversità senza accuratezza o accuratezza senza diversità sono entrambe sbagliate.
- Misure aggiuntive vostre: **spread** (std delle 20 posizioni per pedone, mediata, per orizzonte) e **errore per orizzonte**. Il gaussiano ha lo spread che cresce di più; V2 ha errore minore a ogni orizzonte con meno spread, quindi variabilità messa dove serve.
- Contesto reale: circa 1 m di errore medio, cioè un paio di volte lo spazio occupato da una persona. Si capisce in che zona della banchina sarà qualcuno, non il punto esatto.

8.5 Metriche distribuzionali (14b)

- **Velocità:** bimodale (fermi ≈ 0, in cammino ≈ 1.25 m/s), riprodotta da tutti; V2 ha un po' troppi pedoni fermi.
- **Accelerazione:** forma giusta; V2 più liscio della realtà (la media delle ultime 3 velocità è già liscia e il modello ci resta vicino).
- **Tortuosità:** quasi tutti i percorsi sono dritti; V1 un po' più curvo, V2 un po' più dritto.
- **Distanza dal vicino:** tutti i modelli generano coppie sotto circa 0.4 m, che nei dati reali non esistono. È la proprietà più difficile, perché richiede di coordinare i futuri di pedoni diversi. Cause possibili: grafo fisso al frame 9 (chi si avvicina dopo non è collegato), media dei messaggi (non conta la densità), nessun termine esplicito di repulsione.

Da dire all'interview (EN): "why do your models generate pedestrians that are too close?"

"Avoiding collisions requires coordinating the futures of different pedestrians. Our model does see the neighbours' generated positions at every sampling step, but the graph is fixed at the last observed frame, so two people who only come close later are not connected. The mean aggregation also hides how crowded the neighbourhood is, and nothing in the loss penalises overlap explicitly. Rebuilding the graph from the generated positions during sampling, or adding a distance-based guidance term at inference, would be the next things to try."

8.6 11a: bozza (la risposta è vuota in ANS)

Da sistemare prima della consegna: 11a – bozza, circa 190 parole

Pedestrian motion is intrinsically uncertain: the observed 5 seconds do not determine where a person wants to go, whether they will stop to wait for a train or speed up to catch one, or on which side they pass an obstacle or another person. The future is therefore multimodal. A model trained with MSE would predict the average of these options, for example a path straight through an obstacle, or a slow walk that is an average of standing and walking. A probabilistic model can generate distinct, realistic futures and represent their spread. Compared with part one: (1) the uncertainty here is genuinely aleatoric (unobserved intentions), whereas the cylinder flow is deterministic and its uncertainty comes from lost information and phase sensitivity; (2) the system is Lagrangian: positions are part of the state, the number of agents varies and there is no fixed mesh, so the graph must be built from proximity; (3) there are no known governing equations; (4) the data are real tracking measurements with limited precision, not simulations; (5) the model conditions on a history of 10 frames and predicts 20 frames at once, instead of one Markov state and one step.

8.7 Altri punti caldi della parte 2

Da dire all'interview (EN): 9b/9c: "why only permutation symmetry?"

"The order of pedestrians in the array carries no meaning, so permuting them leaves the scene unchanged, and our model, message passing plus per-pedestrian attention, is permutation equivariant by construction. Translations, rotations and reflections are not symmetries of this data, because the coordinates refer to a fixed platform with walls, rails, stairs and a kiosk at fixed places. Moving a trajectory would put people on the rails. Time is ordered, so permuting frames destroys the velocities."

Da dire all'interview (EN): 11d: "why time bundling instead of autoregression?"

"Three reasons. Cost: a generative model already needs 20 sampler steps per prediction, so autoregressive frame-by-frame generation would need 20 times 20 calls per future. Error accumulation: errors compound over 20 autoregressive steps, as we saw in assignment 1, while a bundle has no compounding inside the 10 seconds. Fit to the data: pedestrians walk smoothly towards a goal, so generating the whole path at once matches the data. The price is a fixed horizon and a graph that cannot be rebuilt inside the bundle."

Da dire all'interview (EN): 12d: "1000 pedestrians or 300 timesteps?"

"With k-nearest neighbours the number of edges grows linearly: 8,000 edges for 1,000 people instead of about a million for a fully connected graph, and the temporal attention is per pedestrian, so that is also linear in N. Three hundred timesteps is harder: temporal attention is quadratic in the sequence length, so 100 times more work, which still fits on one GPU but could be made linear with windowed attention. The bigger problems are not computational: the learned frame embedding has a fixed size of 30, a graph fixed at the last observed frame becomes meaningless over a long horizon, and generating 300 frames at once is a much harder task."

Da dire all'interview (EN): 14d: "what is a non-obvious limitation?"

"The loss is averaged over all pedestrians in a batch, so a scene with 100 people weighs ten times more than a scene with 10, and the model mostly learns crowded behaviour. And because positions are stored with limited precision and velocities come from differences of two frames, even standing people have a small random velocity, which our informed prior partly interprets as unpredictability."

Punto debole / trappola: domande scomode probabili sulla parte 2

- **Why only 10,000 of 100,000 samples?** Budget di calcolo (T4, tempo), e le curve si appiattiscono dopo circa 5 epoche: probabilmente più dati avrebbero aiutato più di più epoche. Siate coerenti con la 14e ("we have plenty of data").
- **Why the last-epoch model and no early stopping?** Nessun overfitting visibile (val vicina a train) e l'ADE di validation oscillava molto (160 scene): scegliere l'epoca migliore su una misura rumorosa sarebbe stato quasi casuale.
- **Why does V2 have the highest loss but the best results?** Il target $x_1 - x_0$ dipende dal prior, quindi le loss di modelli con prior diversi non si confrontano (D2).
- **Mean vs sum aggregation?** La media rende i messaggi indipendenti dal numero di vicini (con kNN è sempre k, tranne nelle scene piccole), ma nasconde la densità della folla.
- **Is the informed prior leaking the future?** No: usa solo i frame 0–9 e viene campionato allo stesso modo in training e inferenza. Quindi $x_0 \perp x_1 \mid c$.

9 Confronto tra le due parti (domande "trasversali")

	Parte 1 (cylinder flow)	Parte 2 (pedoni)
Formulazione	Euleriana: campo su nodi fissi	Lagrangiana: posizioni come stato
Origine dell'incertezza	informazione persa, fase dei vortici, errore di modello (epistemica)	intenzioni non osservate, multimodalità (aleatoria)
Grafo	dato dalla mesh (lati dei triangoli)	costruito: kNN $k = 8$ al frame 9
Aggregazione	somma	media
Asse temporale	Markov, uno stato $\rightarrow$ stato a +1 s, rollout autoregressivo di 6 passi	storia di 10 frame $\rightarrow$ 20 frame in un colpo (time bundling)
Mixing globale	attenzione su tutti i nodi della mesh	attenzione sul tempo, per pedone; lo spazio solo tramite 4 salti di message passing
Embedding di τ	canale scalare concatenato all'input	sinusoidale $\rightarrow$ MLP, sommato a ogni token
Prior	$\mathcal{N}(0, I)$	$\mathcal{N}(0, I)$ vs informato (V1, V2)
Confronto chiave	deterministico vs generativo	due generativi con prior diversi
Metriche	statistica dinamica (wake activity), non puntuale	puntuali (ADE/FDE, $\min_{20}$) + distribuzionali (istogrammi)
Larghezza / parametri	64 / 0.2 M	128 / 1.2 M
Hardware	laptop Apple (MPS)	Tesla T4

Da dire all'interview (EN): "why did you embed τ differently in the two parts?"

"The two parts were developed by different members, and both approaches are standard. In part 1 we followed Practical 3 and concatenated τ as an input channel, the simplest option. In part 2 we used a sinusoidal embedding followed by an MLP and added it to every token, which gives the network a richer representation of the noise level at every position in the sequence. If we unified them, we would use the part 2 approach, or adaptive layer norm, in both."

Da dire all'interview (EN): "what is common to both parts?"

"In both parts the model is a conditional transport model: it learns a velocity field that moves samples from a simple source distribution to the conditional distribution of the future, trained with the conditional flow-matching loss on straight interpolation paths and sampled with 20 Euler steps. In both, the architecture combines local message passing over a graph with attention for the long-range part, and the loss is only applied to the variables we actually predict: the dynamic nodes in part 1, the future frames in part 2."

Da dire all'interview (EN): "what did you learn from comparing the deterministic and the probabilistic model?"

"That with an MSE loss, a deterministic model is not wrong but answers a different question: it predicts the conditional mean. When the future is uncertain, the mean is a smooth field that never actually occurs, and in an autoregressive rollout this smoothing compounds until the wake stops moving. A generative model trained with flow matching uses an MSE loss too, but on local directions in noise-to-data space, so its samples stay sharp. It matches the statistics of the dynamics, not the exact phase. The metric has to be chosen accordingly."

10 Banca domande (risposte brevi in inglese)

Allenatevi a voce, a turno, con il cronometro: 30–60 secondi per risposta. In grassetto la domanda, sotto la risposta minima da dare. Le domande con $\star$ sono le più probabili.

10.1 Teoria generale del corso

T1. $\star$ **What is a neural surrogate and why use one?**
A network trained on simulation data to approximate the evolution map of a system. Training is expensive but amortised over many cheap evaluations, and the surrogate can take much larger time steps than a numerical solver.

T2. **Eulerian vs Lagrangian?**
Eulerian observes fields at fixed points in space; the state is $u(x, t)$, as in our mesh. Lagrangian follows material elements; positions are part of the state, as with the pedestrians.

T3. $\star$ **Invariance vs equivariance?**
Invariance: the output does not change when the input is transformed, $f(gx) = f(x)$. Equivariance: the output transforms in the corresponding way, $f(gx) = gf(x)$. A network is equivariant if every layer commutes with the group action.

T4. **Why does symmetry matter?**
Data efficiency, consistency under changes of coordinates, fewer unphysical predictions: the model does not have to learn the symmetry from data.

T5. $\star$ **Why is message passing permutation equivariant?**
The same message function is applied to every edge, the same update to every node, and messages are aggregated with a permutation-invariant operation such as a sum. Relabelling the nodes only relabels the outputs.

T6. **Is self-attention permutation equivariant?**
Yes, without index-based positional encoding: permuting the tokens permutes Q, K and V, and $\mathrm{softmax}(PSP^\top) = P\,\mathrm{softmax}(S)P^\top$, so the output is permuted the same way.

T7. **Does permutation equivariance give rotation equivariance?**
No. Those are different groups. Feeding coordinates or relative vectors into a generic MLP does not guarantee equivariance to rotations or translations.

T8. **How does EGNN achieve E(n) equivariance?**
Messages depend only on invariant quantities such as squared distances, and coordinates are updated by scalar weights times relative vectors $x_i - x_j$, which rotate and translate correctly.

T9. $\star$ **Why do autoregressive errors grow?**
The next prediction uses the previous prediction, so $e_t \approx J_t e_{t-1}$: the error is amplified by the model's sensitivity to its input, plus new error at each step. For a stochastic model it behaves at least like a random walk, growing like $\sigma\sqrt{t}$.

T10. **Teacher forcing vs push-forward?**
Teacher forcing always feeds true states during training, so the model never sees its own errors. Push-forward feeds the model's own prediction as input and trains it to still reach the true next state, which makes it robust to its own error distribution.

T11. **What is time bundling? Pros and cons?**
Predicting several future steps in one call. Fewer calls and less error accumulation, at the price of a harder task, a fixed horizon, and no way to update interactions inside the bundle.

T12. $\star$ **Why does direct regression from noise to data fail?**
The optimal predictor under MSE is $\mathbb{E}[X_1 \mid X_0]$. With noise independent of the data this is just $\mathbb{E}[X_1]$: every noise sample maps to the mean, so the distribution collapses.

T13. $\star$ **What does the flow-matching regressor learn instead?**
$v^*(x, t) = \mathbb{E}[X_1 - X_0 \mid X_t = x]$: the average *local* direction at a point and time, not the average

destination. Integrating this field transports the noise distribution into the data distribution without collapse.

T14. **What is the continuity equation and why does it appear?**
$\partial_t p_t + \nabla \cdot (p_t v_t) = 0$. It describes how a density evolves when samples are moved by a velocity field. It is what links the velocity field we learn to the probability path from noise to data.

T15. **Why is flow matching "simulation free"?**
Training only needs interpolated points x_t and the target $x_1 - x_0$, both in closed form. No ODE is solved during training, only at sampling time, unlike maximum-likelihood training of continuous normalising flows.

T16. **Normalising flows: what limits them?**
Invertibility constraints on the architecture, expensive Jacobian determinants, dimension preserved through all layers, and limited expressivity per layer.

T17. **What is a probability path?**
A time-indexed family of distributions p_t with p_0 the source (noise) and p_1 the data. Flow matching specifies it through conditional paths such as straight lines between a noise sample and a data sample.

10.2 Parte 1: architettura e dati

A1. ⋆ **Walk me through your architecture.**
Each mesh becomes a graph with nodes as vertices and edges as triangle sides in both directions. Nodes get normalised velocity, normalised position and a one-hot node type; edges get the relative vector and its length. MLP encoders map these to 64-dimensional latents. Four blocks follow, each with one MeshGraphNets message-passing step and one global self-attention layer, both residual. An MLP decoder outputs the normalised 1-second velocity increment per node.

A2. ⋆ **Why add attention to message passing?**
Because of the 1-second horizon. In one second the flow moves about one unit in a 1.6-long channel, while four message-passing steps only reach about 0.15, less near the cylinder where edges are short. With about 600 nodes, full attention is cheap and gives a global receptive field in one layer.

A3. **Why predict the increment rather than the next velocity?**
The increment is about five times smaller than the field, so after normalisation it is a unit-scale target, and the network does not have to copy the large, mostly unchanged part of the field. It is also the analogue of an Euler step, as in MeshGraphNets.

A4. **How do you handle different mesh sizes in a batch?**
We pad every batch to one of two fixed sizes, 640 or 1024 nodes, with masks so that padded nodes and edges send no messages, are ignored by attention and have zero weight in the loss. Fixed shapes avoid kernel recompilation on the GPU.

A5. **Where do the cylinder and the Reynolds number enter the model?**
Implicitly. The cylinder is given by the wall nodes and their coordinates, and the inflow speed by the velocities of the inflow nodes, which together set the flow regime.

A6. **How are the boundary conditions handled?**
The node type is an input; the loss and the noise only act on dynamic nodes; the increment is only added on dynamic nodes; after every rollout step the inflow and wall values are restored.

A7. **Why normalise per component, and with which statistics?**
u_y is much smaller than u_x, so without per-component scaling the loss would ignore u_y. All statistics come from the training split only and are reused unchanged for validation and test.

A8. **How many parameters, and where are they?**
197,506 for the deterministic model, 197,698 for flow matching (three extra input channels). About 93% are in the four processor blocks.

10.3 Parte 1: flow matching e training

F1. ⋆ **Write down your flow-matching training step.**
x_1 = normalised increment on dynamic nodes, $x_0 \sim \mathcal{N}(0, I)$ on dynamic nodes, $\tau \sim U[0,1]$ once per sample, $x_\tau = (1-\tau)x_0 + \tau x_1$. The network gets the conditioning features plus x_τ and τ, and the loss is the masked MSE between its output and $x_1 - x_0$.

F2. ⋆ **How do you sample?**
Start from fresh Gaussian noise on the dynamic nodes, then take 20 explicit Euler steps $x \leftarrow x + \frac{1}{20} v_\theta(x, \tau \mid C_t)$ from $\tau = 0$ to 1. De-normalise, add to the current velocity and restore the boundaries.

F3. **Why is the noise zero on boundary nodes?**
Their increment is zero by definition, so there is nothing to model there. Leaving noise on them would only add inputs the network has to learn to ignore.

F4. **One τ per node or per sample?**
Per sample: the whole field is at the same point of the path. A different τ per node would not correspond to the probability path we integrate at sampling time.

F5. **How do you make the comparison fair?**
Same network class and size, optimiser, schedule, number of steps, and the same sequence of training pairs through the same random seed. The only differences are the three extra input channels and the loss.

F6. **How did you select the checkpoint?**
Lowest loss on a fixed set of validation pairs after each epoch: epoch 19 for the deterministic model, epoch 28 for flow matching. The test set was used only for the final evaluation.

F7. **What would you change in training first?**
Add input noise or push-forward training against exposure bias, train longer, and select the model on a rollout metric on validation instead of the one-step loss.

10.4 Parte 1: risultati

R1. ⋆ **Summarise your results in two sentences.**
The deterministic model's wake activity collapses from 0.05 to 0.005 because it predicts the conditional mean and the rollout converges to an almost static, time-averaged flow. Flow matching keeps a stable activity around 0.045, about half of the ground truth's 0.10, and its samples show vortex shedding with a plausible but different phase.

R2. **Why is FM still below the ground truth?**
Some samples lose their dynamics during the rollout, probably because of exposure bias. Training was short, Euler integration with 20 steps smooths a little, and τ only enters at the input. The median is much lower than the mean, so a subset of rollouts goes static.

R3. **Why is the ground-truth standard deviation so large?**
The trajectories have different inflow speeds and geometries, hence different Reynolds numbers. Some have a nearly steady wake and others strong shedding. It is physical variability, not noise.

R4. **Why exclude the first second?**
The initial state is a transient: the flow is still developing and changes everywhere, even upstream. We want to evaluate the statistically steady shedding.

R5. **Why average the metric over samples and not the samples themselves?**
Averaging the fields would recreate the smooth conditional mean we wanted to avoid. The metric is a property of each realisation, so we compute it per sample and then average.

R6. **Could a model that outputs random noise score well on this metric?**
Yes, that is a weakness of the metric: it rewards any change. That is why we also checked the rollouts visually. Ours stays below the ground truth and the samples look physical.

10.5 Domande trabocchetto

X1. **"Your deterministic validation loss is 2.5; a constant predictor gets about 2. Is your model useless?"**
Two things. The value 2 holds on the training distribution; the validation trajectories may contain larger increments, since the gap is already there at epoch 1, but we did not compute the baseline on validation. But it is also the point of the assignment: after one second the conditional mean carries little pointwise information. The rollouts show the model does learn the mean flow structure, just not the shedding.

X2. **"Navier–Stokes is deterministic, so why not make the deterministic model bigger?"**
A bigger model would reduce the reducible part of the error but not the conditional variance caused by unresolved information and phase sensitivity; under MSE the optimum is still the conditional mean. Only a model of the distribution can produce sharp samples.

X3. **"Your FM validation loss is three times lower than the deterministic one, so it is three times better?"**
No, the losses are different objectives on different targets. The CFM loss includes the noise and has a positive minimum. Models must be compared on the rollouts.

X4. **"Did you tune anything on the test set?"**
No. Statistics from train only, checkpoint selected on validation, the test set used once with the supplied evaluator.

X5. **"Did you run the step-count experiment?"**
We tried, but it ran out of GPU memory on the laptop, so our answer is based on theory: one step gives the conditional mean, more steps restore diversity until it saturates.

X6. **"What if I reorder the nodes of the mesh?"**
The output is reordered in the same way: message passing and attention are permutation equivariant, and positions are features, not index-based encodings.

X7. **"What if I shift the whole domain by one metre?"**
Edge features would not change, but the normalised positions would fall outside the training range, so the prediction would change. The model is not translation invariant because the problem, a fixed channel, is not.

X8. **"Why not a diffusion model?"**
Diffusion is closely related: it is flow matching with a different, curved Gaussian path and, usually, stochastic sampling. Flow matching with straight paths is simpler, trains with a plain regression and tends to need fewer steps. It is also what the course and the assignment prescribe.

11 Cheat sheet (da rileggere prima di entrare)

Tre formule

- MSE $\Rightarrow f^*(c) = \mathbb{E}[Y \mid c]$, loss minima $= \mathbb{E}\,\mathrm{tr}\,\mathrm{Cov}(Y \mid c)$
- CFM: $x_\tau = (1-\tau)x_0 + \tau x_1$, target $x_1 - x_0$, ottimo $v^* = \mathbb{E}[x_1 - x_0 \mid x_\tau]$
- 1 passo Euler: $x_0 + v^*(x_0, 0) = \mathbb{E}[x_1 \mid c]$ (= deterministico)

Altre da ricordare

- equivarianza $f(gx) = gf(x)$; invarianza $f(gx) = f(x)$
- MPNN $h_i' = \gamma(h_i, \bigoplus_j \phi(h_i, h_j, e_{ij}))$
- continuità $\partial_t p_t + \nabla \cdot (p_t v_t) = 0$
- errore AR $e_t \approx J_t e_{t-1} (+\xi_t)$; RMSE $\sim \sigma\sqrt{t}$
- EGNN: messaggi da $\|x_i - x_j\|^2$, update $\sum (x_i - x_j)\zeta(m_{ij})$

Parte 1 – dati

- 500/20/20 traiettorie, 600 stati, $\Delta t = 0.01$ s, orizzonte 1 s = 100 stati
- 250k coppie train; $\approx$ 600 nodi (552–898)
- dinamici = interni (0) + outflow (5); bordi = inflow (4), wall (6)
- dominio 1.6×0.41; arco medio 0.037
- du_std $\approx (0.10, 0.064)$ vs u_std $\approx (0.51, 0.115)$

Parte 1 – modello

- nodo: $\tilde{u}(2)$ + pos(2) + tipo(4) = 8; FM $+x_\tau(2)$ $+\tau(1) = 11$
- arco: $(x_j - x_i)/\bar{\ell}$ (2) + norma (1)
- 64 hidden, 4 blocchi (MP + MHA 4 teste), somma, residual, LN
- 197,506 / 197,698 parametri
- target $\Delta\mathbf{u}$ normalizzato; MSE solo su nodi dinamici
- AdamW 1e-3, cosine, wd 1e-4, clip 1.0, batch 8, $30 \times 200 = 6000$ step
- $\approx$8 s/epoca, $\approx$4 min, Apple MPS
- best: det epoca 19 (val 2.525), FM epoca 28 (val 0.786)
- Euler 20 passi; rumore e target zero sui bordi; un τ per campione

Parte 1 – wake activity (media)

	1–2	2–3	3–4	4–5	5–6
GT	.142	.109	.101	.099	.098
Det	.049	.013	.008	.006	.005
FM	.094	.051	.043	.047	.046

Mediana FM $\approx$ 0.02–0.04 dopo il primo intervallo: molti rollout si spengono.

Parte 2 – modello

- 10k scene: 7k/1k/2k, seed 42
- kNN $k = 8$ al frame 9; messaggi $\mathrm{MLP}(h_i, h_j, x_j - x_i)$, media
- 4 blocchi: attn temporale + MP spaziale + FFN; hidden 128; 4 teste
- frame emb. appreso + tipo obs/gen + τ sinusoidale$\rightarrow$MLP
- AdamW 3e-4, wd 0.01, batch 16, 20 epoche, $\approx$1.2 M parametri, T4
- 20 Euler, 20 futuri; 73 ms/chiamata (32 scene)
- V1: ultima vel. + rumore $\propto$ std obs (min 0.1)
- V2: media ultime 3 + rumore crescente con l'orizzonte

Parte 2 – risultati [m]

	ADE	FDE	mADE	mFDE
Gauss	1.059	2.213	0.751	1.457
V1	1.061	2.152	0.791	1.522
V2	.987	1.994	.722	1.372

Frasi chiave

- "The deterministic model is not wrong, it predicts the conditional mean."
- "Flow matching averages local directions, not final destinations."
- "The CFM loss has a positive minimum and is not comparable to the MSE."
- "One Euler step gives back the deterministic model."
- "A sample with the wrong phase is still a valid future."
- "We did not test this, but from the theory we would expect…"